



Cold Start Latency in Serverless Computing: A Systematic Review, Taxonomy, and Future Directions

MUHAMMED GOLEC, School of Electronic Engineering and Computer Science, Queen Mary University of London, London, United Kingdom of Great Britain and Northern Ireland and Abdullah Gul University, Kayseri, Turkey

GUNEET KAUR WALIA, Department of Information Technology, Dr BR Ambedkar National Institute of Technology, Jalandhar, India

MOHIT KUMAR, Department of Information Technology, Dr BR Ambedkar National Institute of Technology, Jalandhar, India

FELIX CUADRADO, Polytechnic University of Madrid Higher Technical School of Computer Systems Engineering, Madrid, Spain

SUKHPAL SINGH GILL, School of Electronic Engineering and Computer Science, Queen Mary University of London, London, United Kingdom of Great Britain and Northern Ireland

STEVE UHLIG, School of Electronic Engineering and Computer Science, Queen Mary University of London, London, United Kingdom of Great Britain and Northern Ireland

Recently, academics and the corporate sector have paid attention to serverless computing, which enables dynamic scalability and an economic model. In serverless computing, users only pay for the time they actually use resources, enabling zero scaling to optimise cost and resource utilisation. However, this approach also introduces the serverless cold start problem. Researchers have developed various solutions to address the cold start problem, yet it remains an unresolved research area. In this article, we propose a systematic literature review on cold start latency in serverless computing. Furthermore, we create a detailed taxonomy of approaches to cold start latency, which we use to investigate existing techniques for reducing the cold start time and frequency. We have classified the current studies on cold start latency into several categories such as caching and application-level optimisation-based solutions, as well as Artificial Intelligence/Machine Learning-based solutions. Moreover, we have analyzed the impact of cold start latency on quality of service, explored current cold start latency mitigation methods, datasets, and implementation platforms, and

Muhammed Golec thanks the Ministry of Education of the Turkish Republic, for their support and funding. F. Cuadrado has been supported by the HE ACES project, Spain (Grant No. 101093126).

Authors' Contact Information: Muhammed Golec, School of Electronic Engineering and Computer Science, Queen Mary University of London, London, United Kingdom of Great Britain and Northern Ireland and Abdullah Gul University, Kayseri, Turkey; e-mail: m.golec@qmul.ac.uk; Guneet Kaur Walia, Department of Information Technology, Dr BR Ambedkar National Institute of Technology, Jalandhar, Punjab, India; e-mail: guneetkw.it.22@nitj.ac.in; Mohit Kumar, Department of Information Technology, Dr BR Ambedkar National Institute of Technology, Jalandhar, Punjab, India; e-mail: kumarmohit@nitj.ac.in; Felix Cuadrado, Polytechnic University of Madrid Higher Technical School of Computer Systems Engineering, Madrid, Comunidad de Madrid, Spain; e-mail: felix.cuadrado@upm.es; Sukhpal Singh Gill, School of Electronic Engineering and Computer Science, Queen Mary University of London, London, United Kingdom of Great Britain and Northern Ireland; e-mail: s.s.gill@qmul.ac.uk; Steve Uhlig, School of Electronic Engineering and Computer Science, Queen Mary University of London, London, United Kingdom of Great Britain and Northern Ireland; e-mail: steve.uhlig@qmul.ac.uk.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 0360-0300/2024/11-ART65

<https://doi.org/10.1145/3700875>

classified them into categories based on their common characteristics and features. Finally, we outline the open challenges and highlight the possible future directions.

CCS Concepts: • **Computer systems organization** → **Distributed Computing**; *Cloud Computing*; • **Networks** → Network Latency;

Additional Key Words and Phrases: Serverless computing, cold start latency, function as a service, survey, cloud computing

ACM Reference Format:

Muhammed Golec, Guneet Kaur Walia, Mohit Kumar, Felix Cuadrado, Sukhpal Singh Gill, and Steve Uhlig. 2024. Cold Start Latency in Serverless Computing: A Systematic Review, Taxonomy, and Future Directions. *ACM Comput. Surv.* 57, 3, Article 65 (November 2024), 36 pages. <https://doi.org/10.1145/3700875>

1 Introduction

With the developments in virtualization technology, the concept of cloud computing has witnessed developments that abstract responsibilities such as server maintenance and infrastructure management from customers and put them under the responsibility of service providers [1]. In light of these developments, one of the latest developments that integrates **Backend as a Service (BaaS)** and **Function as a Service (FaaS)** services is serverless computing [2]. The first example of serverless computing is the Lambda platform introduced by Amazon in 2014 [3]. In this service model, code developers write their code as functions. These functions are processed by triggers such as **Hypertext Transfer Protocol (HTTP)** in serverless computing operating in event-driven architecture. In this way, customers can focus only on code development by abstracting from server management processes such as server and container management. In addition to this benefit, it has other advantages such as serverless computing, dynamic scalability, and a pay-as-you-go economic pricing model [4]. This means that resources can automatically scale up and down based on demand, and are only charged for the time the resources are used.

Literature reported that interest in serverless computing is growing rapidly and its market size will reach \$22 billion by 2025 [5]. The same research results reported that by 2025, around 50% of global businesses could use serverless computing. Serverless computing, along with developing processor and sensor technologies, shapes technology by being used in a wide variety of sectors and fields. One of these areas is edge computing, where latency and bandwidth performance are increased by bringing the processing power closer to the data source [6]. To benefit from the advantages of serverless computing and edge computing, researchers have focused on the serverless edge computing paradigm, where these two concepts are integrated. This means lower latency and bandwidth consumption, as well as isolating infrastructure management from the customer and automatically scaling resources [7]. Additionally, since data is processed on edge devices, this means lower security and privacy issues [8]. Research conducted by *MarketsandMarkets* shows that the total Serverless edge computing market will be approximately 100 billion dollars by the end of 2029 with the development of 5G and **Internet of Things (IoT)** technologies [9].

In addition to the advantages it offers, serverless computing also brings with it challenges such as security and privacy concerns, resource management, and the cold start latency that are still waiting to be solved [10]. One of the most important of these challenges is the cold start. In the serverless paradigm, idle resources are released due to energy efficiency and favorable pricing policy. This process is known as scaling to zero [11]. Resources scaled to zero take some time to be processed for reuse when a new function arrives. This latency is called a cold start in serverless computing [12]. Cold start latency has negative effects such as delayed response to time-sensitive applications, inconsistent performance, increased resource consumption, and poor user experience [5]. Handling this challenge is essential for a smooth and responsive user experience. In addition,

it will contribute to the rapid adaptation of companies to this service model by increasing confidence in serverless computing. As per our best knowledge, no systematic review and taxonomy study has been conducted for the cold start latency problem in serverless computing before. A comprehensive systematic review is the basis for understanding a research area and research gap. In addition, such a study can provide an overview by presenting up-to-date solutions for interested researchers and academicians.

1.1 Motivation and Contributions

Serverless computing is one of the latest service models of cloud computing [13]. Compared to other service models, it has attracted attention with its advantages such as rapid auto-scaling, abstraction of infrastructure management from the customer, and economic model [14]. In addition to these advantages, it also brings challenges such as cold start that are still waiting to be solved. When the literature is examined, it is tried to solve the cold start problem with the studies carried out in both the academy and the private sector. As far as we know, there is no Systematic Review and Taxonomy study examining cold start studies in serverless computing in the literature. This survey aims to present research gaps and contribute to the development of serverless computing by providing researchers with an overview of the cold start problem. This article makes three main contributions to a better understanding of cold start and cold start current solutions in serverless computing as follows:

- We are bringing the most comprehensive and largest systematic review and taxonomy study to date on cold start and cold start solution studies in serverless computing to the literature. To do this, the authors of this article systematically analyzed over 100 papers from academia and industrial literature, technical reports, book chapters, and reference lists. With these analysis results and the snowball technique applied, a total of 32 paper sets containing cold start solutions were obtained (Section 4).
- One of the most important aspects of a systematic review is to identify research questions [15]. We define five research questions (Section 4.2) to review and analyze the current literature on the cold start problem in serverless computing. In line with these questions:
 - Cold start and QoS relationship (Section 5.1),
 - Factors affecting cold start (Section 5.2),
 - Existing literature studies and classification about cold start solutions (Section 5.3),
 - Platforms used by studies about cold start solutions (Section 5.4),
 - The places where cold start studies are published and the dataset/code availability are analyzed (Section 5.5).
- The clear challenges of cold start in serverless computing are discussed and future directions are identified, providing valuable information for future researchers (Section 6).

1.2 Article Organization

Figure 1 shows the organization of this article. In Section 2, the relevant survey studies in the literature are explained and compared. Section 3 explains the basic concepts of serverless computing and cold start, including the cloud delivery model, serverless architecture, serverless platforms, serverless applications, and **Quality of Service (QoS)** parameters. Section 4 explains how the research papers were collected and the research questions for this article. In Section 5, the data obtained from the collected papers for each research question is analyzed and solution-based classification is made. In the Section 6, open challenges and future directions regarding cold start are highlighted for prospective researchers. Finally, Section 7 concludes the article.

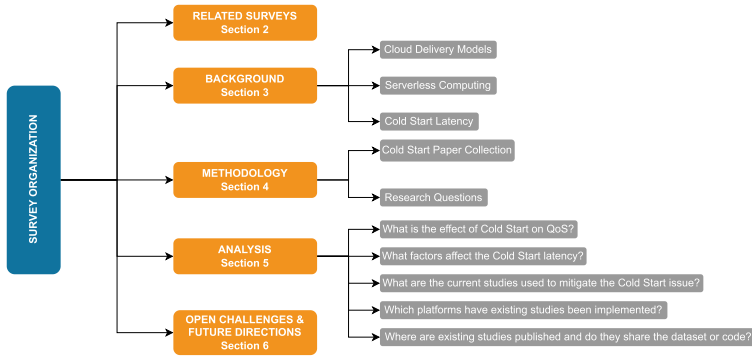


Fig. 1. Organization of the survey.

2 Related Surveys

With the popularity of serverless computing, taxonomy and survey studies on serverless computing have been conducted in recent years, examining various aspects of it. In this section, we provide an overview of the contributions and limitations of current literature studies.

Vahidina et al. [16] investigated cold start mitigation strategies in serverless computing in their study. Further, they also conducted experiments on cold start targeting a commercial platform. In these experiments, they determined the number of cold starts for concurrent requests, sequential requests, and concurrent with different time intervals requests, respectively. Mampage et al. [17] have done a review that examines the concept of resource management in serverless computing from multiple perspectives. They examined in detail the factors affecting resource management, workloads, and system design in serverless computing. They also analyzed the literature on resource management in serverless. In another research study [18], serverless architecture was discussed in detail as four stack layers. These layers are virtualization, capsule, system orchestration, and system coordination, respectively. The authors analyzed the contribution of related studies to resource management for each layer and highlighted the challenges. The authors of References [19, 20] examine serverless applications in their literature study. The authors of Reference [19] classify serverless implementations in eight main areas, discussing the viability of serverless and the challenges encountered. They search for all solutions by classifying the challenges they detect under nine headings among themselves. In the research in Reference [20], 16 features are examined in serverless applications. Answers are sought for questions such as in which situations serverless applications are advantageous and how they should be applied. For this, by applying the mixed method 89 papers are examined. The authors compare 10 existing studies in the literature to confirm the validity of the research results. Jawaddi and Ismail [21] have proposed a taxonomy for autoscaling in serverless. They compared the studies on serverless autoscaling in the literature and found that the literature studies are still insufficient in terms of scaling target scope and **Service-level Agreement (SLA)** requirements. And they also identified that there is still a need for models such as anomaly and energy-aware autoscaling in this area as a research gap. The authors of References [5, 22–24] have proposed survey studies to identify research gaps and opportunities in serverless computing. The authors of References [22, 24] compared serverless platforms and tools, emphasizing the advantages and disadvantages. The authors of Reference [23] reviewed challenges and solutions related to infrastructure characteristics in serverless. They also provide a literature review to identify the challenges and opportunities found in serverless. The study in Reference [5] covers 17 different research directions, including performance optimization and multi-cloud development on serverless. All solutions for 17 research directions are compared.

Cassel et al. [25] extensively examine the use of the IoT in serverless computing in their review. For this, 60 different articles were reviewed and the main challenges for incubating and executing functions were examined. And solutions to these challenges, such as protocols and programming languages, were explored. In Reference [26], an in-depth analysis of security in serverless computing was done. The authors examine serverless architectures, highlighting security shortcomings and possible solutions. Dittakavi's paper is far from a review and taxonomy and only provides a limited summary of cold start-related research [27]. Additionally, it does not include future research directions and a comprehensive analysis of cold start solutions. Mallick et al. [28] conducted a systematic review on network security challenges and solutions in serverless computing. He reported that attackers are leveraging cold start to cause service interruptions and reduce system performance, which can lead to **denial-of-service (DoS)**. Likewise, Ahmadi [29] examines the relationship between cold start and DoS attacks in his short review study examining network security challenges and solutions in serverless computing. Donta et al. [30] They conducted a survey to provide a deep understanding of **distributed computing continuity systems (DCCS)**. This survey discusses the evolution of computing paradigms up to DCCS. Additionally, the opportunities and limitations (such as cold start) offered by each computing paradigm (cloud, edge, etc.) are examined. The authors of Reference [31] examined the limitations of protocols used for edge networks and ways to facilitate learning in edge networks. Huang et al. [32] address mobile communication technologies and quality wireless services in the survey. They emphasize that ML algorithms can be used to improve the efficiency and energy optimization of QoS parameters in communication systems. The same ML algorithms can be used to optimize the performance degradation in QoS parameters that occur due to cold start in serverless computing. Nazari et al. [33] conducted a survey study examining optimization efforts in serverless computing. The survey examines new strategies such as isolation techniques and intelligent algorithms to solve the cold start problem. In addition, solutions that increase the processing efficiency of functions executed in a container are also examined.

2.1 Critical Analysis

Table 1 compares our work with existing surveys is given. The “*” symbol indicates just an overview, the “+” symbol indicates comprehensive discussion, and the “–” indicates the absence of the feature. They are also abbreviations for **CSR: Cold Start Relevance**, **FRD: Future Research Directions**, and **CFCs: Comprehensive Analysis for Cold Start Solutions**. While some of these articles cover the cold start issue superficially, to our knowledge, no previous work has focused on the cold start issue in detail and an in-depth analysis of its solutions. This survey provides a comprehensive analysis of the studies found in the literature to solve cold start problem. It also provides an overview of the cold start issue in serverless and the factors affecting cold start. This article brings different technical approaches into a clear framework by categorizing them into a detailed taxonomy.

3 Background

In this section, we explain the background concepts related to serverless computing and cold start problem, as shown in Figure 2.

3.1 Cloud Models

Cloud Models are examined under two subsections. The first subsection is cloud computing models according to deployment. The second subsection is cloud computing models according to delivery.

3.1.1 Cloud Deployment Models. There are four cloud deployment models shown in Figure 3 based on their different features and characteristics, such as public cloud, private cloud, hybrid

Table 1. Comparison of our Systematic Literature Review with Existing Surveys

Work	Topic	Year	Type	Publisher	CSR	FRD	CFCs
[16]	Cold Start	2020	Review	IEEE	+	—	—
[17]	Resource Management	2022	Taxonomy	ACM	*	+	—
[18]	Architecture	2022	Survey	ACM	*	+	—
[19]	Applications	2022	Survey	ACM	*	+	—
[20]	Applications	2022	Survey	IEEE	*	*	—
[21]	Autoscaling	2023	Taxonomy	Research Square	*	+	—
[22]	General	2021	Survey	Springer	*	*	—
[23]	Infrastructure	2023	Survey	IEEE	*	*	—
[25]	IoT	2022	Review	Elsevier	*	*	—
[24]	General	2021	Review	arXiv		*	—
[5]	General	2023	Review	ACM	*	+	—
[26]	Security	2022	Review	Springer	—	—	—
[27]	Cold Start	2023	Review	Eduzone	+	—	—
[28]	Security	2024	Review	World Scientific News	*	—	—
[29]	Security	2024	Review	IJCSRR	*	+	—
[30]	General	2023	Review	MDPI Computers	*	—	—
[31]	Data Protocol	2023	Review	IEEE EDGE	—	+	—
[32]	Mobile Communication	2018	Survey	Springer MNA	—	—	—
[33]	Serverless Platforms Optimization	2021	Survey	IEEE	*	+	*
This Work	Cold Start	2024	Review, Taxonomy	ACM	+	+	+

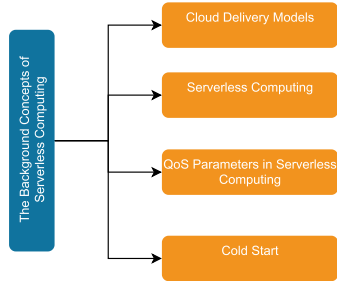


Fig. 2. The background of serverless computing and cold start.

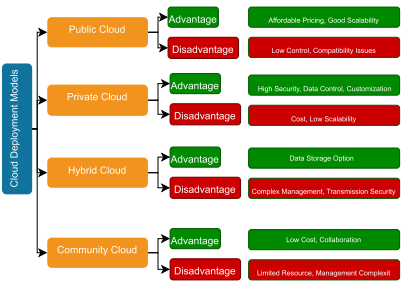


Fig. 3. Cloud deployment models.

cloud, and community cloud. Furthermore, Section A of Appendix A provides a detailed explanation of these models.

3.1.2 Cloud Delivery Models. There are four cloud Delivery models shown in Figure 4 based on their different features and characteristics, such as **Infrastructure as a Service (IaaS)**, **Platform as a Service (PaaS)**, **Software as a Service (SaaS)**, and **Function as a Service (FaaS)**. Furthermore, Section B of Appendix A provides a detailed explanation of these models.

3.1.3 Serverless Computing Architecture. A general architecture of serverless computing is shown in Figure 5. The functionality of this paradigm can be demonstrated via its following components:

- **Event Sources:** These include sources such as database change, file upload, HTTP requests, scheduled timers, and so on. Although this paradigm was initially proposed for the cloud, however, it has positioned itself in IoT use cases by bringing functions closer to the devices [34]. It comprises a wide range of application scenarios, such as ubiquitous phones,

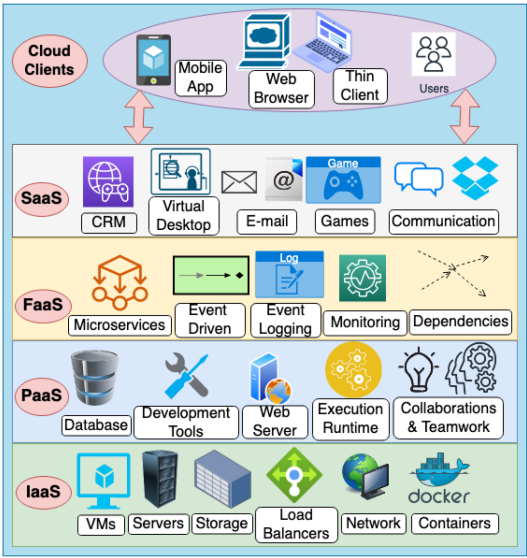


Fig. 4. Cloud delivery models and their services.

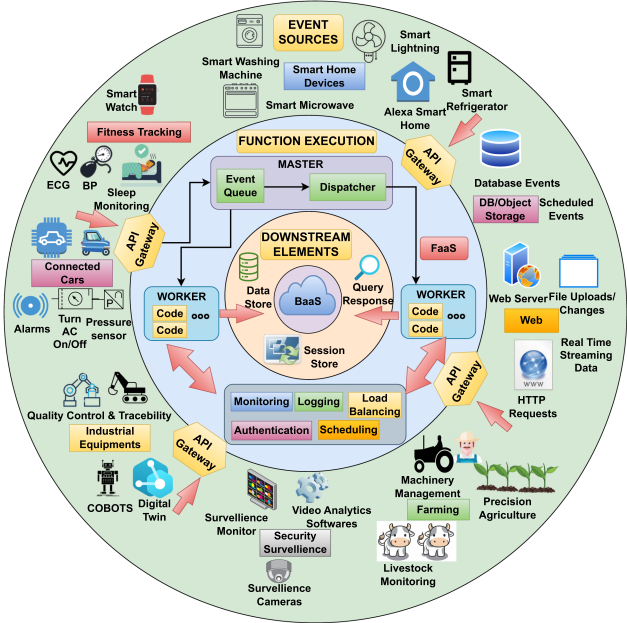


Fig. 5. Architecture and workflow in serverless computing.

smart home devices (amazon echo, ring doorbell, etc.), fitness tracking (monitoring blood sugar levels, heart rate, recording quality sleep), industrial equipment (Collaborative Robots known as COBOTS), connected cars (turning on alarms on driver sleep detection, applying brakes, accelerating), farming (Livestock monitoring, precision agriculture), and nevertheless streaming application data, which yields time-series data as depicted in Figure 5. All

these applications thrive for an apt paradigm that can provide real-time responsiveness and agility to incorporate an escalating number of IoT equipment and the corresponding surge in data volume.

- *API Gateway*: It acts as an interface between the event sources, which can be called frontend, and the Function Execution layer providing FaaS [35]. It is responsible for routing the incoming request to specific serverless functions. Apart from this, it performs some pre-processing operations such as request validation, aggregating requests into batches, protocol transformation, and response formatting before forwarding it back to the consumer/requesting device.
- *Function Execution*: This component defines the core of serverless computing architecture, with the advent of which the responsibility of application deployment, management, scalability, and data security can be offloaded to cloud servers [36]. On the occurrence of an event, the request is forwarded to the event queue via API gateway, which further aims to invoke pertinent functions as per set defined rules. Then, the scheduler is responsible for dispatching the incoming request to the appropriate worker node, which provides a suitable environment comprising required resource requirements. To complete the function execution, the required application code is loaded from the repository. Finally, the user receives the response once execution is complete after which created environment is destroyed and corresponding resources are released [17].
- *Downstream Elements*: It comprises the elements such as data store, session store, query response, and so on, which are triggered in response to the execution of a serverless function [6]. These elements are generally invoked as a result of the completion or successful execution of a serverless function.
- *Backend-as-a-Service (BaaS)*: Although the cloud remains an indispensable part of serverless computing, however, the details of infrastructural functionality, scalability and management remain a black box for the serverless computing consumer [37]. These characteristics of serverless enables offloading of the database server and application logic to the cloud thus enabling the professionals to hone in on **Business Logics (BL)**.

3.1.4 Serverless Computing Platforms. Serverless platforms are generally examined under two main headings: open source and commercial. The classification of serverless platforms is given in Figure 7, and a detailed review of these platforms is given in Section C of Appendix A.

3.1.5 Serverless Computing Applications. The use cases for this architectural paradigm span various domains ranging from web applications and APIs, providing effortless scaling of business to IoT applications, requiring real-time data processing as shown in Figure 6, which shows how these applications are utilizing **Artificial Intelligence (AI)** to optimize the serverless computing-based service.

Taxonomy of Serverless Applications: Further, we have proposed a detailed taxonomy of serverless applications. These applications are discussed below:

- **Real-time analysis of Geospatial data:** This data comprises addresses, maps, **Satellite Imagery (SI)**, coordinates (longitudes, latitudes), and so on, relating to different geographical and topographic phenomena. Such data constitutes an aggregation of physical features, which requires a framework for the perception and cognition of features. Apart from optimizing latency, **Serverless Computing (SC)** improves the processing in the case of high-dimensional hyper-spectral data [38]. Mete et al. [39] deployed **Geographic Information Systems (GIS)** applications to the serverless platform AWS Lambda. As a result of the tests, it was seen that GIS applications on serverless platforms respond faster under high workloads.

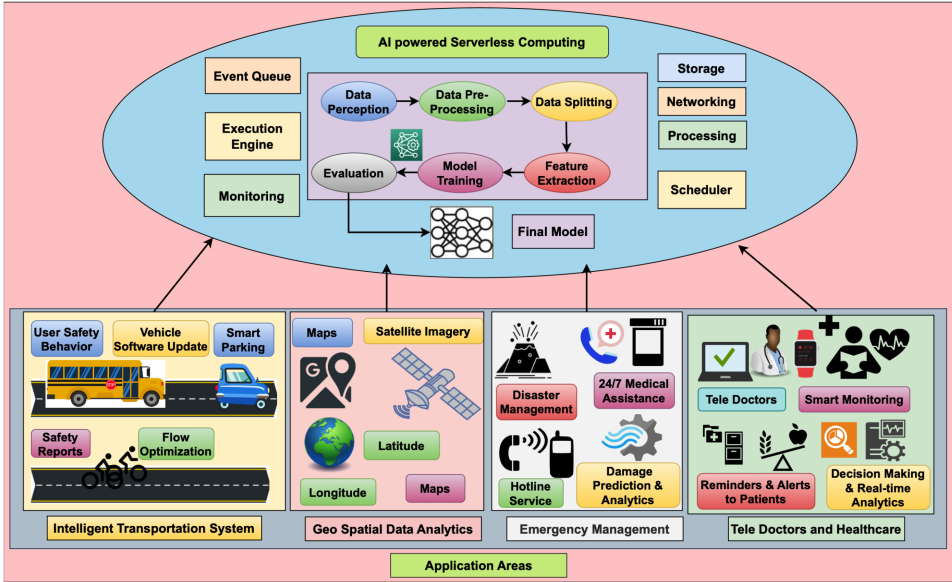


Fig. 6. AI-powered serverless computing applications.

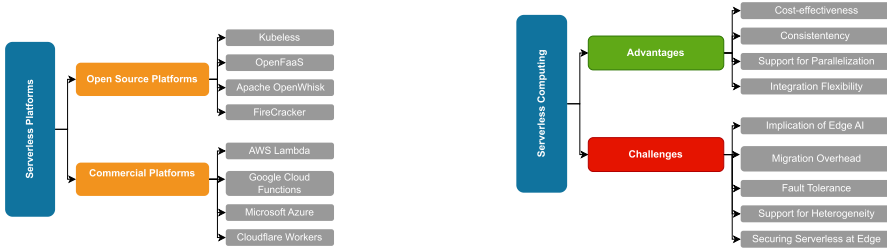


Fig. 7. Taxonomy of serverless platforms.

Fig. 8. Summary of advantage and challenge in serverless computing.

- **Emergency Management:** Some use-cases such as disaster management, identifying damage extent, handling pandemic situations such as delivering essentials to affected zones, and emergency hotlines require scalable resources, and **Business Intelligence (BI)** capabilities to coordinate relief operations. Serverless extends all cloud-based functionalities, streamlines deployment, and handles dynamic workloads, thus making data processing much more convenient [19]. Franz et al. [40] proposed a system that aims to quickly reunite children who were lost after a disaster with their families, with the IoT-based system called ReuniFam. Families and rescue teams use Raspberry, an IoT device located in shelters, to upload images of missing children to a website. If there is any match between the uploaded images and images from sources such as drones, then the location of the missing children is updated in the system. The authors used AWS Lambda, a serverless platform, for fast face recognition time.
- **Intelligent Transportation System (ITS):** The increasing trend of autonomous vehicles thrives for capabilities such as vehicle route planning, smart parking, emergency braking, weather incorporation, fleet management, and so on. Hence, leveraging SC capabilities can result in a more agile and responsive ITS. For instance, this computing paradigm has

Table 2. Comparison of Existing Works Based on the Taxonomy of Serverless Applications

Work	Application Type	Platform
[39]	Real-time Analysis of Geospatial Data	AWS Lambda
[40]	Emergency Management	AWS Lambda
[41]	Intelligent Transportation System (ITS)	AWS Lambda, GCP
[10]	Telemedicine and Healthcare	GCP
[43]	AI and ML-based Enterprise Data Intelligence	AWS Lambda
[45]	Blockchain-based Serverless Applications	GCP
[47]	IoT-based Serverless Applications	NA

been implicated in the calculation of the **Origin/Destination (O/D)** matrix for **Bus Rapid Transit (BRT)** utilizing Bluetooth signals [41].

- **Telemedicine and Healthcare:** SC enables organizations to deliver healthcare services in a more personalized manner, empowering appointment scheduling, remote diagnosis, virtual health assistants, bots, and most importantly tele doctors, enabling patients to get medical assistance regardless of their physical whereabouts. For example, Golec et al. [10] proposed a serverless-based framework that provides early Covid diagnosis for an increasing number of patients.
- **AI and ML-based Enterprise Data Intelligence:** It provides an ideal platform for executing AI models optimizing resource allocation, and leveraging event-driven architecture [42]. It allows the execution of AI algorithms cost-effectively, as resources are provisioned only when required. Jefferson et al. [43] recommended a system sentiment analysis using **Natural Language Processing (NLP)** techniques. To take advantage of the processing power and affordability of serverless computing, they deployed the Naïve Bayes model to AWS Lambda, a serverless platform.
- **Blockchain-based Serverless Applications:** Blockchain-based applications have problems such as high network congestion due to the number of users and transaction volume [44]. Serverless could be a potential solution to this problem with dynamic scalability property. In addition, blockchain applications (smart contracts, etc.) work with trigger logic based on events such as transactions and data changes. Serverless’s event-drive architecture is suitable for these applications. Golec et al. [45] proposed a serverless-based framework that enables early COVID detection. They have integrated a blockchain module to enable the integration of patients’ sensitive data such as health information and biometric data.
- **IoT-based Serverless Applications:** Since serverless platforms can scale with increasing demand, they are suited for IoT applications with varying workloads and data amounts [46]. In addition, it can provide an economical model for IoT applications with sudden workloads thanks to the pay-as-you-go model. When the literature is examined, many IoT-based serverless applications can be seen. Benedict [47] proposed a new framework using IoT-based serverless and a societal application that monitors air quality for smart cities.

Table 2 shows a comparison of existing works based on the proposed taxonomy of serverless applications and platforms.

3.2 Quality of Service (QoS) Parameters in Serverless Computing

The following are discussions of these QoS parameters:

- **Latency:** It is defined as the time delay between consumer request submission and receiving the response from the corresponding serverless function [14, 48]. This time is resultant

of request queuing time, request-to-resource mapping time, and finally the serverless function execution time. Hence, reducing the cold start delay in resource set-up significantly optimizes this parameter making it suitable for real-life use cases.

- *Throughput*: It is measured by the number of requests processed in unit time by the serverless system. This metric is considered effective as it helps in determining the overall system efficiency along with the underlying system's capability to handle high arrival rate workload [49, 50]. Consequently, due to the negative correlation between mean latency and system throughput, techniques used to reduce the latency, ultimately ameliorates the system's overall throughput.
- *User Experience (UX)*: The agile infrastructure of serverless computing not only reduces cost but also eases the hassle of administrative overhead [51]. The **Service Providers (SPs)** aim at delivering positive UX via optimizing warm-up strategies, periodic monitoring and analyzing the performance metrics,
- *Cost*: The billing and pricing model offered by a particular serverless platform decides the function execution cost [14]. Due to its pay-as-you-go pricing model, the cost is significantly determined by the function execution time. Apart from the service provider context, the cost is determined by the number of functions requested and resources utilized for the function execution time span. Hence, this parameter becomes of crucial consideration by both service providers and service consumers.
- *Scalability*: Serverless paradigm possesses the capability of automatically provisioning resources as and when desired by the incoming workloads. Scalability can be vertical or horizontal. The horizontal one is characterized by the addition and elimination of existing containers, whereas the vertical scaling aims at elevating (adding more cores) or reducing (removing cores) capabilities of existing containers [52].
- *Resource Utilization*: The autonomous resource handling capabilities of serverless make it a suitable choice for serving geographically distributed edge/fog nodes. After application deployment on the serverless platform, the service provider becomes accountable for managing the whole chain of **resource management (RM)** activities such as resource provisioning, scheduling, and final allocation along with monitoring and resource scaling [17, 53].

3.2.1 Serverless Computing Advantages and Challenges. In this section, we explain the advantages and challenges of serverless computing, which are summarised in Figure 8.

The advantages of serverless computing are discussed below:

- *Cost-effectiveness*: This computing offers lower cost as compared to traditional hosting, as the consumer needs to pay for specific amounts of time, hence there is no upkeep cost during idle times. In contrast to conventional VM (which come with a hefty expense of duplicating data along with OS and underlying application environment), serverless offer lightweight abstractions in the form of unikernels such as microVMs, which automatically scale the resources as per incoming requests [54, 55].
- *Consistent for IoT Applications*: Serverless platform provides fine-grained scalability and resource provisioning, which suits well for spiky workloads of IoT use cases [56, 57].
- *Support for Parallelized Execution*: it equips applications such as **Defence IoT (DIIoT)** with the capability to carry on computations in a parallelizable fashion. For instance, whenever any hazard occurs, prompt and independent task processing capabilities are required for equipment to be traced, to ensure timely response and at the same time not affecting other mission-critical operations.
- *Flexibility to be integrated with Cloud-Edge Computing*: The latest research is trending toward creating a collaborative cloud-fog-edge environment to cater to real-world application

scenarios. Although serverless was originally designed for the cloud, however, to ameliorate edge capabilities, Serverless-powered edge platforms are emerging such as AWS IoT Greengrass [58].

The challenges of serverless computing can be listed as follows:

- *Implication of Edge AI*: Despite its numerous advantages, the practical implication of Edge AI in serverless voices serious challenges, as training and testing ML models at the network edge in the computation finite resources environment of serverless computing becomes I/O intensive [34]. It contradicts the fact that serverless is designated to work well with CPU-bound functions, which ultimately makes it cost-effective [53]. In fact, this computing aims to minimize resource usage, both in execution as well as in idle states.
- *Migration Overhead*: The adoption of vendor-specific proprietary APIs, runtime environments, and other native services makes it challenging to migrate with another vendor. Migration becomes a tedious task as it involves differences in billing models, reconfiguring infrastructure, and significant changes in code.
- *Fault Tolerance*: The feature of fault tolerance ensures that the running application remains available and responsive even in case of container fail over.
- *Support for Heterogeneous Hardware*: the platforms support for heterogeneous hardware such as **Graphics Processing Unit (GPU)**, and **Field Programmable Gate Arrays (FPGAs)** remains a challenging task.
- *Securing Serverless at Edge*: Although serverless is backed by firewalls and a **Trusted Execution Environment (TEE)** at the cloud layer, security gets a backseat when serverless computing is bought at the edge [35]. It occurs because of distributed nature of microservices and multi-tenancy amongst services, further, the deployment of functions in a containerized manner makes it lesser isolated in comparison to VMs [59].

3.3 Cold Start Problem

In serverless computing, each function is assigned to a separate container for the execution of functions [12]. If the container is ready, then the function is directly assigned to this container, and the function is executed. However, if there is no ready container, which is mostly the case, then it is necessary to prepare a new container to execute the function. In the serverless paradigm, containers are released after a specific time (τ) after executing functions to avoid wasting resources. This process is known as the scale to zero [23]. After the containers are scaled to zero, it is necessary to start the container again for the requests coming to the server. Starting a new container and preparing the function for execution causes a specific latency. In the serverless paradigm, this latency period is called a cold start [13]. The main causes of cold start are as follows:

- Since there is no ready container when the first request comes to the server, it is necessary to start a new container. Preparatory processes will take time, such as starting a new container and loading the runtimes and libraries needed to execute the function. This causes a cold start.
- Serverless paradigm automatically scales resources if needed thanks to its dynamic scalability feature. Sudden increases in requests can cause scalability delays. This delay is another reason why cold start occurs.
- In serverless platforms, resources (CPU, RAM, etc.) are shared between functions. Moreover, resource problems such as resource contention caused by spikes in demand can cause cold start latency [4].

Figure 10 shows the steps from creating a container to executing a function to illustrate the cold start process. These processes are generally the same on all platforms.

- *Loading and Preparation Phase*: When a function request is sent to the platform, the container should be prepared to execute this function. The platform first parses the headers and input parameters from the incoming data. It then allocates the necessary resources, such as CPU and RAM, to execute the function. To execute the function, a container is started, and the necessary dependencies, such as libraries and runtime, are loaded into the container. Function code is deployed to the container and made ready for execution.
- *Function Execution*: After the container loading and preparation phase is completed, the function assigned to the container is executed. In running the function, the platform monitors and manages the necessary resource management. External service interactions of the function with the network, such as the API, occur in this process. After the execution process is completed, the response is prepared and sent to the client. Resources are kept warm for τ to respond to new requests and prevent a cold start; this process is known as a warm start. If there is no request during this time, then the containers are scaled to zero, saving energy.

4 Review Methodology

In this section, we discuss the review methodology used to conduct this systematic review of cold start latency in serverless computing.

4.1 Cold Start Papers Collection

In this section, the evaluation process followed to collect the studies on the cold start problem in serverless computing is mentioned. Serverless computing is still a new field of research considering it was first introduced in 2014. Therefore, there may be literature publications that are meaningless or beyond the scope of this article. For this, the paper collection process should be carried out with great care. To find the papers examined in this survey; We have benefited from leading digital databases such as IEEE, Springer, ACM, ScienceDirect, and Wiley. Since serverless computing was introduced in 2014, naturally all articles are for after 2014. Keywords used to find related articles are as follows:

$$[(ServerlessComputing) \text{ || } (ColdStart)] \& [(ServerlessComputing) \text{ || } (ColdStart)], \quad (1)$$

$$[(ResourceManagement) \text{ || } (ServerlessComputing)] \& [(ColdStart) \text{ || } (FaaS)], \quad (2)$$

$$[(ColdStartMitigation)] \& [(ServerlessComputing), \quad (3)$$

$$[(ServerlessComputing)] \text{ || } [(ColdStart)]. \quad (4)$$

Using these keywords, a total of over 100 serverless computing and cold start-related materials were obtained from Academic & Industrial Literature, Technical Reports, Book Chapters, and Resources in the Reference List (A). In the second stage (B), the abstract and conclusion parts of the obtained papers were systematically examined. Further, unrelated papers were excluded from the research. To remove potential bias in paper selection and screening, the entire review process (A-B) was carried out jointly among the authors participating in this survey. The 25 papers obtained are directly related to solving cold start in serverless computing. In the last stage, the backward snowball method was applied by an expert author to expand the paper set [60]. Snowball Method is a sampling technique used to collect hard-to-reach data [61, 62]. The expert writer first analyzes a paper about cold start. It then scans all citations in the subsections and reference section of this article. In this way, papers that do not contain the keywords “Cold Start” or “Serverless” but focus on solving cold start in serverless are identified. As a result, the number of papers obtained reached 32 (C). Figure 9 shows the paper collection process for this survey.

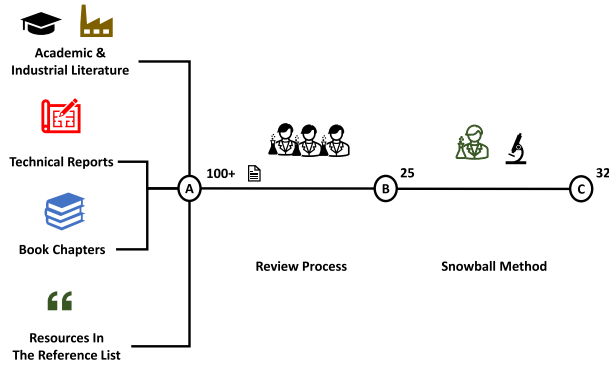


Fig. 9. Paper collection process.

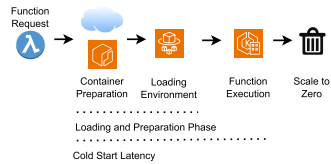


Fig. 10. Cold start latency process.

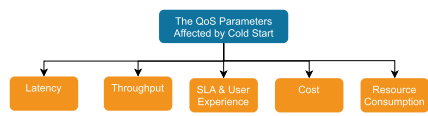


Fig. 11. QoS parameters affected by cold start.

Table 3. Research Questions of the Survey

No	Research Question	Motivation	Section
1	What is the effect of Cold Start on QoS?	The purpose of this research question is to investigate how cold start affects which QoS parameters.	5.1
2	What factors affect the Cold Start latency time?	The purpose of this research question is to determine the factors affecting cold start latency.	5.2
3	What are the current studies used to mitigate the Cold Start issue?	The purpose of this research question is to identify current studies for cold start and to search for current solutions for cold start.	5.1
4	Which platforms are the current studies been implemented?	The purpose of this research question is to identify the platforms where current studies for the cold start are applied and evaluated.	5.4
5	Where are existing studies published and do they share the dataset or code?	The purpose of this research question is to identify publishers that publish current studies for cold start. It is also aimed to explore the reproducibility of the current work.	5.5

4.2 Research Questions

One of the most important aspects of a systematic review is to identify research questions [15]. In this section, we define research questions to examine and analyze current literature on the cold start problem in serverless computing. Research questions, motivation, and relevant section information are given in Table 3.

5 Analysis

In this section, we discuss how we addressed the above-mentioned questions in this study.

5.1 RQ1: What Is the Effect of Cold Start on QoS?

The purpose of this research question is to investigate how cold start affects which QoS parameters. The QoS parameters that cold start can affect are given in Figure 11. The cold start problem caused by the serverless paradigm adversely affects various QoS parameters such as latency, throughput, SLA, and User Experience [63, 64]. Developers need to consider these effects when designing their applications. In this section, we examine how a cold start affects these parameters;

- **Latency:** It is the time taken until the data from the source reaches the destination and comes back, and it is usually measured in **milliseconds (ms)** [14]. It is one of the most frequently used QoS parameters to measure the performance of an application. Golec et al. [45] found high delays due to cold start in a serverless-based application that followed patients instantaneously. They sent an increasing number of concurrent requests to the server to represent an increasing number of users. The results showed that 100 concurrent requests had more latency than 200 concurrent requests (5 ms). The reason for this is the cold start situation that occurs when the first requests (100 simultaneous requests) arrive on the server. Especially in applications such as time-sensitive and instant patient follow-up, cold start may cause unwanted latency.
- **Throughput:** It is a QoS parameter defined as the data transmitted per second over a network or communication channel [14]. To achieve SLA, the use of resources, i.e., Throughput, is important. Sudden increases in server traffic due to cold start cause resource contention and negatively affect throughput. Golec et al. [4] found in their study that there were significant decreases in throughput due to resource contention. They sent varying numbers of workloads (100, 200, 300, 500, and 1000) to the server. The resource contention that occurred due to the cold start in the system (500 requests) reduced the throughput value (from 470 to 430 (P/s)).
- **SLA and User Experience:** It covers all experiences such as accessibility and functionality experienced by users using serverless systems while interacting with the platform [14]. It is one of the important concepts that service providers should pay attention to for customer continuity [65]. Cold start negatively affects User Experience as it causes unwanted latency [66, 67].
- **Cost:** Serverless computing, with a pay-as-you-go model, charges customers only for the time resources are used. However, a cold start may mean more charges as it will increase the execution time of the function [12]. Also, some platforms like **Google Cloud Functions (GCF)** and **AWS Lambda** [68] implement keep-warm strategies that require containers to run idle to mitigate the effects of a cold start. This strategy causes additional costs for customers.
- **Scalability:** Serverless computing can automatically scale resources with increasing demand [17]. The prolongation of the delay times due to cold start will also affect the scalability time. This causes performance loss in serverless computing.
- **Resource Consumption:** High cold start frequency may cause non-optimal allocation in case of creating new instances as it will make busy the system's resources such as CPU and RAM.

5.1.1 Optimized Caching Mechanisms. Optimized caching mechanisms are currently used in almost all serverless platforms to reduce latency and increase efficiency.

- **Impact of optimized caching mechanisms on latency:** Caching data close to where the calculation is performed eliminates the need to retrieve data from remote databases and

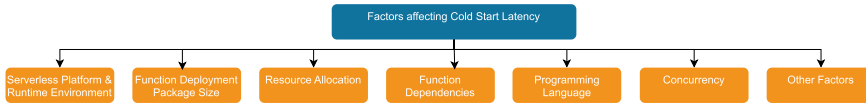


Fig. 12. Factors influencing cold start latency.

enables faster processing of functions. AWS Lambda implements a caching strategy with Amazon ElastiCache service, and experimental results show latency reduction [69]. This service appears as the Memorystore (Redis) service in GCP and it is observed that the latency decreases in the same way with ElastiCache [70]. Azure, however, reduces latency by performing caching operations using Azure Cache [71].

- **Impact of optimized caching mechanisms on throughput:** Thanks to the optimized caching strategy, more simultaneous requests can be responded to and this increases throughput as this means more data. Likewise, it has been reported that ElastiCache, Memorystore, and Azure Cache increase throughput, because they allow more data processing [72–74].

5.1.2 Performance Metrics Used To Evaluate Cold Start Solutions. This subsection explains the basic metrics used to evaluate the effectiveness of cold start frequency reduction and cold start prevention efforts in serverless computing. These metrics are:

- **Resource Consumption:** This metric, usually expressed in units such as RAM consumption (MB) and CPU usage amount (GHz), represents the amount of resources used in the cold start prevention solution (such as starting a new container usage) [14].
- **Cost:** This metric expresses the additional costs that the cold start solutions bring to the system depending on the methods they use [12]. This metric is usually used when evaluating solutions that require additional resource consumption and is represented in USD currency based on the execution cost.
- **Scalability:** This metric expresses the horizontal and vertical scaling performance to evaluate solutions to the cold start problem [68]. The number of containers launched per second can be used as a unit.

5.2 RQ2: What Factors Affect the Cold Start Latency?

The purpose of this research question is to investigate the factors affecting the cold start latency that occurs in serverless computing. Figure 12 shows these factors. Although the factors affecting cold start can be a very wide range, they are generally as follows:

- **Serverless Platform and Runtime Environment:** Each commercial and open-source platform uses different isolation and container runtime techniques specific to its architecture [75]. Therefore, cold start latency times differ on each platform [17]. Li et al.[18] compared cold start latency in various isolation and container runtime environments. The results showed that HyperContainer had the highest cold start latency while Process-based Docker had the lowest cold start latency. In another study [76], the authors compared cold start latency for AWS Lambda and Microsoft Azure. The results showed that the Microsoft Azure platform has a better performance compared to AWS Lambda. A similar comparison is made in Reference [77] for AWS Lambda and Microsoft Azure platforms. Factors such as the software language used on serverless platforms, function dependencies (file size), CPU, and RAM affect the cold start latency period. **Interpreted languages such as Python, JavaScript, and Node.js have shorter cold start latency than compiled languages such as JAVA, C#, and**

.NET [78]. Each platform has optimization options and design and engineering features for different languages [79]. That is why factors such as programming language and function dependencies cause different performance on each platform. For example, AWS Lambda runs the Python language faster than other platforms [80]. In their experiments, Wang et al. compared AWS Lambda, GCF, and Azure for Nodejs [79]. The results confirm that Nodejs creates smaller cold start latency on AWS. Experiments by Lee et al. show similar results [81]. AWS has a faster runtime in Python, Nodejs, C#, and Java languages than, Azure, IBM, and Google. Another factor that affects cold start latency is **function dependencies (file size)**. Golec et al. In their experiments, they calculated the cold start latency of functions with function dependencies of 1 Kb, 16 MB, and 37 MB in size to GCF. **The results show that cold start increases with the size of function dependencies.** There are new trends, such as Pre-Warming and Edge Computing, which will be explained in later sections, that can potentially mitigate these factors more effectively, and programming language-specific optimizations offered by the previously described platforms.

- **Function Deployment Package Size:** With the increase of the Function deployment package, cold start latency is expected to increase [18, 82]. Because it will take longer to load large-size packages into existing containers than small-size packages. To test this, Manner et al. [77] uploaded deployment packages of various sizes to the AWS Lambda platform and calculated cold start amounts. Experiment results showed that the function deployment package had a negative effect on cold start latency.
- **Resource Allocation:** On serverless platforms, resources such as RAM and CPU can be adjusted manually in the environment configuration. Depending on the amount of resources (RAM and CPU), the deployment time of the containers also varies and this time is expected to affect the cold start [22]. The authors of References [4, 18, 77] conducted experiments to examine the relationship between RAM amount and cold start on serverless platforms. The results showed that the cold start time decreased in proportion to the increased amount of RAM. It should be noted that on most serverless platforms the CPU scales linearly with the amount of RAM [82].
- **Function Dependencies:** Before a function can be deployed, dependencies such as libraries required for the function's execution must be loaded into the containers [82]. Since this loading process takes time, it has a direct effect on cold start [83, 84]. To pre-provisioning dependencies for commonly used functions can reduce the number of cold starts [5].
- **Programming Language:** Serverless platforms come with various programming language options such as Python, Java, and C# [20]. Since the software languages have different environment overheads and architectures, the amount of cold start they have will also be different. **High-level languages such as Python and Java have additional startup delays, because they contain language runtimes such as JVM [85]. In References [4, 18, 76, 77] experiments showed that interpretative languages (Python, Java) have higher cold start than compiled languages.**
- **Concurrency:** With the scaling feature in serverless computing, a new container is started for each concurrent request. As a result, peak loads occur due to excessive use of resources and peak loads trigger cold start [77]. Mohan et al. [86] and Ustiugov et al. [67] conducted experiments on two different platforms to observe the effect of concurrent on cold start. The results show that cold start increases with increasing concurrency.
- **Other Factors:** The other factors affecting cold start can be:
 - If the function is dependent on the data in the storage, then the storage access process will increase the cold start time [68],

- Designing a function to run in a **Virtual Private Cloud (VPC)** increases cold start latency due to network isolation functions [82],
- Depending on the geographical region infrastructure where the function is distributed, cold start latency may be affected [87].

5.3 RQ3: What Are the Current Studies Used to Mitigate the Cold Start Problem?

The purpose of this research question is to identify and search current studies for cold start. To answer RQ3, the first two authors carefully examine the contents of the 32 papers available and then obtain the research summary. Then, the solutions obtained from the papers are classified under the necessary subheadings. Figure 13 shows the taxonomy of the current techniques used to mitigate the cold start problem.

It is important to solve the cold start problem to increase the performance of QoS parameters and serverless applications. One of the biggest problems caused by cold start issues is latency and it is seen that some studies have been done in the literature to solve this problem. Literature studies are generally examined under two main headings. These; (i) studies to reduce the cold start latency period and (ii) studies to reduce the frequency of cold start.

5.3.1 Studies to Reduce Cold Start Latency Time. The studies to reduce cold start latency are generally aimed at speeding up container preparation and loading function libraries. The studies can be examined under 4 subheadings. Table 4 shows the studies and methods used to reduce cold start latency time. It should be noted that these subheadings may vary according to the review methodology:

- **Cache-based Solutions:** Serverless computing uses application delivery technology containers that provide isolated environments as runtime environments. Cache-based solutions rely on preloading required or commonly used libraries into containers. In this way, it is aimed to speed up the container preparation process and shorten the cold start latency period. Mohan et al. [86] proposed a solution based on pre-rendering network creation to speed up container provisioning. They drastically reduced cold start latency by loading networks into paused containers using a system they call a **Pause-Container Pool Manager (PCPM)**. In another study, Liu et al. [88] recommended FaaSLight, a new approach based on app optimization. **FaaSLight** reduces cold start latency by loading only the core part of the application code, which is one of the root causes of cold start. In the study suggested by Wang et al. [89], with the **replayable execution optimized method**, the image of the applications can be saved and shared between containers. In this way, applications are restored if necessary, thus shortening the application start-up time and thus the cold start latency period. In Reference [90], new scheduling methods are proposed, an approach that reduces latency for chain functions (functions that call each other). After the first function is called, the runtime environment for the next function to be called is predicted and prepared in advance. In this way, the cold start latency time can be reduced. In another similar study, Daw et al. [91] proposed a tool called **Xanadu**, which provides resources when needed to reduce the cold start that occurs in chained functions. Cache-based solutions can be inspiring to reduce cold start latency time, but determining an effective cache policy may not be as easy as it seems. Also, most of the cache-based solutions are those that require high resource consumption for the runtime environment [5]. Cache-based solutions vary on each platform in terms of implementation and effectiveness. This is due to differences in the architecture and optimization techniques of each serverless platform. Since each platform has its own strengths and trade-offs, it is most logical to decide on the platform to be used according to the project. Amazon, Microsoft, and Google serverless services use Amazon ElastiCache, Memorystore,

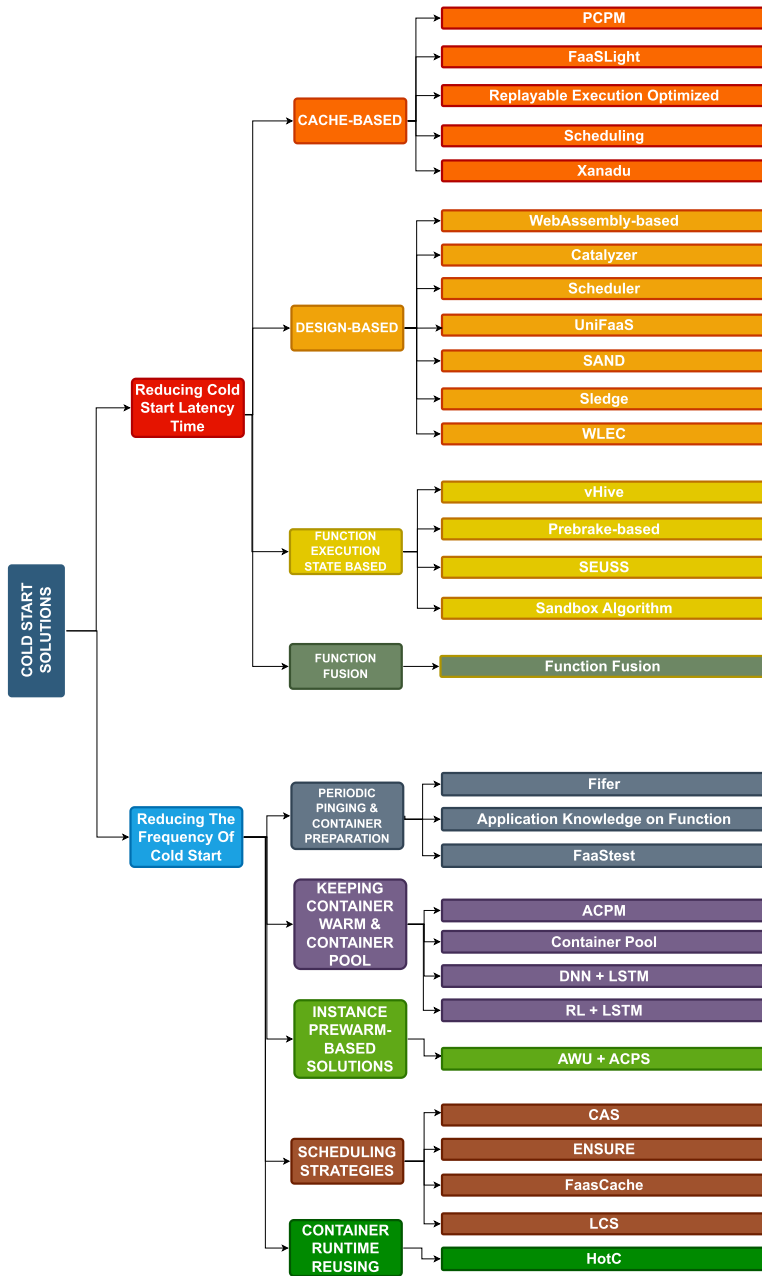


Fig. 13. Taxonomy of cold start latency time reduction techniques and cold start frequency reduction techniques.

and Azure Cache for Redis to speed up access to applications [92–94]. The cache-based solutions interact with the underlying architecture of serverless computing, its differences, and tradeoffs in terms of implementation and effectiveness were examined in detail in Section A of Appendix B.

Table 4. Classification of Cold Start Latency (CSL) Time Reduction Techniques

Solution Method	Objective	Work	Technique Used
Cache-based	CSL	[86]	PCPM
		[88]	FaaSLight
		[89]	Replayable Execution Optimized
		[90]	Scheduling
		[91]	Xanadu
		[95]	WebAssembly-based
Design-based	CSL	[85]	Catalyzer
		[84]	Scheduler
		[96]	UniFaaS
		[83]	SAND
		[97]	Sledge
		[98]	WLEC
Function Execution State-based	CSL	[67]	vHive
		[105]	Prebrake-based
		[106]	SEUSS
Function Fusion	CSL	[82]	Sandbox Algorithm
		[107]	Function Fusion

— **Design-based Solutions:** Studies on improving isolation and runtime architectural designs to reduce cold start latency in serverless computing. Hall et al. [95] introduced a new terminology study that leverages WebAssembly as an alternative to containers for serverless functions. First, serverless access models are characterized to determine runtime requirements, and then performance comparisons are made with the Openwhisk platform to show that WebAssembly is a good alternative. It has been found that the proposed run has less cold start latency than containers. The authors of Reference [85] proposed Catalyzer, a new sandboxing architecture with strong isolation and fast function initialization. Catalyzer restores the function instance from the checkpoint image to reduce the initialization cost. Thus, it can reduce the cold start latency time. The study suggested by Vedaiei et al. [84] proposed a new scheduler in serverless computing that eliminates queue imbalances and also reduces interference. Using The scheduler, only function code and its dependencies (libraries, etc.) can be processed in the sandbox without the need for a full operating system image. Thus, the cold start latency time can be reduced. In Reference [96], the authors introduced UniFaaS as a serverless platform for edge devices. They suggested using unikernels against the container technology used as runtime on serverless platforms. And they reported that UniFaaS has much lower latency for both cold and warm starts. Akkus et al. [83] introduced a new serverless platform, SAND. By introducing a new sandboxing approach, SAND was able to reduce the container preparation process and thus the cold start latency time. The authors of Reference [97] presented a new serverless framework Sledge for edge based on WebAssembly. Sledge has fast startup times by supporting high-density computation, so it shows great promise in reducing cold start latency. In Reference [98], a new container architecture WLEC has been proposed. WLEC uses S2LRU++ Cache replacement policies, thereby speeding up the container preparation process and reducing cold start latency. Oakes et al. [99] proposed a new container system called SOCK. SOCK works by optimizing workloads (Avoiding Kernel scalability bottleneck, Implementing Zygotte provisioning strategy) and experiments have shown that SOCK is 2.8× faster

than AWS Lambda. Li et al. [100] proposed a new lightweight container runtime called RunD. RunD uses MicroVM to provide high-density deployment and high-concurrency startup. The authors adopted RunD as an Alibaba serverless container and reported in their experiments that RunD had the performance of starting over 200 containers per second. Shillaker et al. [101] have introduced a new isolation abstraction called Faaslet. Faaslet aims to reduce resource load by supporting memory sharing between functions. The runtime in which Faaslet is implemented is **Function-as-a-Service Microservice (FAASM)**. FAASM is a working environment that increases the performance of serverless functions. FAASM can reduce startup time and therefore cold start latency by restoring Faaslet from snapshots. These three studies examined use different runtime environments: container, MicroVM, and WASM, respectively. For this reason, each runtime has its own characteristics and performs differently in different optimizations. Among the runtime environments examined, WASM has the fastest startup time, MicroVM has the strongest isolation, and containers have the best performance for lightweight applications. Containers; it has (i) resource management optimization [102], (ii) image optimization to reduce container image size. MicroVMs; it has optimizations such as (i) snapshotting to speed up booting, (ii) memory ballooning for memory management [103]. WASM; it uses strategies such as (i) eliminating dead code for code optimization, (ii) caching WASM modules to speed up future executions [104]. As a result, each isolation and runtime architectural design has different characteristics. And design-based solutions can be extended in future work, not only for cold start solutions but also for safety and performance studies. Design-based Solutions vary depending on the architecture and design of the serverless platform on which they are applied. Design-based solutions interact with the underlying architecture of serverless computing, its differences, and trade-offs in terms of implementation and effectiveness were examined in detail in Section B of Appendix B.

- **Function Execution State-based Solutions:** In this method, the execution state for a function is stored locally, and if it is called again, the platform starts the function quickly using the stored state. In this way, it is aimed to reduce the cold start latency time. Ustiugov et al. [67] have proposed a new framework vHive that proposes to run an image of a function stored on disk instead of starting a function from scratch. The authors identified the main reason for the cold boot delay as restoring an image of a function populating the guest memory. For this reason, they succeeded in reducing cold start latency by 3.7 times by pre-fetching the working set of pages created when a function is called for the first time to the guest memory. The authors of Reference [105] suggested restoring functions from a previously saved image using a prebrake-based method. In this way, it is aimed to reduce the cold start latency time by reducing the function process start-up time. The study suggested by Cadden et al. [106] introduced a new system-level method called SEUSS that enables the fast distribution of functions. Functions for this are deployed from unikernel snapshots. In this way, it is expected that the cold start latency time will decrease. In Reference [82], authors propose an approach to reduce cold start latency by assigning similar functions to the same containers using the sandbox algorithm. While function execution state-based solutions are successful in reducing cold start latency, they may not be practical for situations such as when the input data payload is very different. Because in such cases, the runtime has to wait for the input data and the function execution state to be loaded. Function Execution State-based Solutions vary depending on the architecture and design of the serverless platform on which they are applied. Function execution state-based solutions interact with the underlying architecture of serverless computing, its differences, and tradeoffs in terms of implementation and effectiveness were examined in detail in Section C of Appendix B.

Table 5. Classification of Cold Start Frequency (CSF) Reduction Techniques

Solution Method	Objective	Work	Technique Used
Periodic Pinging and Container Preparation	CSF	[108]	Fifer
		[109]	Application Knowledge on Function
		[110]	FaaStest
		[82]	ACPM
Keeping Container Warm and Container Pool	CsF	[113]	Container Pool
		[114]	DNN + LSTM
		[68]	RL + LSTM
Instance Prewarm-based Solutions	CSF	[115]	AWU + ACPS
		[116]	CAS
		[117]	ENSURE
Scheduling Strategies	CSF	[118]	FaaSCache
		[119]	LCS
Container Runtime Reusing	CSF	[120]	HotC

- **Function Fusion-based solutions:** This solution method combines sequential functions and aims to remove the cold start of the second function. Lee et al. [107] suggested a Function Fusion-based method to reduce cold start latency. If two sequential functions are combined into one function, then the cold start from the second function is eliminated and there is only one cold start delay. In fusion-based operations, even if the cold start of the second function is eliminated when parallel functions are combined, in serverless computing, the processing time may increase, because the functions are run sequentially. Therefore, a new fusion-based method for parallel functions is still an area of research. Function Fusion-based Solutions vary depending on the architecture and design of the serverless platform on which they are applied. Function fusion-based solutions interact with the underlying architecture of serverless computing, its differences, and tradeoffs in terms of implementation and effectiveness were examined in detail in Section D of Appendix B.

5.3.2 Studies to Reduce the Frequency of Cold Start Occurrence. Studies to reduce the cold start frequency can be examined under five subheadings. Table 5 shows studies and methods used to reduce the frequency of cold start occurrence. It should be noted that these subheadings may vary according to the review methodology.

- **Periodic Pinging and Container Preparation:** In this method, containers are prepared in advance to meet new requests on the serverless platform by following a specific period or workload, and cold start formation is prevented. Gunasekaran et al. [108] recommended Fifer, a new resource management framework that provides cluster efficiency and efficient container utilization. Load forecasting is done using the **Long Short-term Memory (LSTM)** model in Fifer and containers are proactively prebuilt. The authors of Reference [109] suggested a system that prepares containers in advance using application knowledge on function. In Reference [110], an ML-based cost and performance optimization strategy called FaaStest is proposed. FaaStest learns the behavioral models of the service provider and recommends the most suitable model to the user in terms of cost and performance. It also pre-provisions containers for functions by making a time series-based prediction to prevent cold start, predicting when requests are called. The period to follow for pinging can be obtained by analyzing factors such as function CPU usage and request time. However, analyzing these factors and training the AI models used for ping requires much effort. Also,

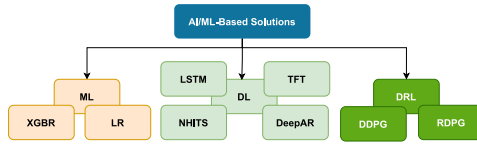


Fig. 14. AI-based solutions.

unnecessary pinging can result in unnecessary resource wastage as containers run idle. Further, AI-based solutions are developed, such as the ATOM and MASTER frameworks [111, 112], in which the request patterns coming to the server are predicted using time-series models as shown in Figure 14. These models predict the occurrence of cold start using **Deep Reinforcement Learning (DRL)** and **Deep Learning (DL)** methods. Thus, they aim to lay the foundation for future cold start prevention studies that do not waste resources (resource-sensitive). The authors chose DRL and DL methods, because these two models are successful in solving complex and nonlinear problems. However, there are still challenges to be overcome in the integration of solutions based on AI/ML models into serverless computing environments: (i) Due to the nature of the serverless computing paradigm, a cold start delay occurs when functions are launched for the first time. AI models with high startup requirements (RAM, CPU, large amount of data, etc.) can further increase this latency. (ii) Serverless platforms do not contain large amounts of resources, because they work using functions that trigger each other. For this reason, AI models that require high processing power may lead to problems such as execution errors and reduced scaling performance in serverless environments. (iii) One of the most important features of serverless computing is the pay-as-you-go pricing model. Since AI models will consume a high amount of resources, they may cause the user not to benefit from the pay-as-you-go advantage. In this solution method, ping can be sent to the server in cases where a cold start does not occur. This may cause unnecessary waste of resources. For this reason, it is very important to increase the prediction accuracy of the AI models used (such as using new models). The DRL models examined within the ATOM framework are **Deep Deterministic Policy Gradient (DDPG)** and **Recurrent Deterministic Policy Gradient (RDPG)**. Experiments have shown that these models have some advantages and disadvantages compared to each other in terms of prediction performances, computation time, and CO_2 emission amount. The RDPG model is 6 times slower than DDPG. This is because **Recurrent Neural Networks (RNNs)** add complexity to the model to increase the learning performance of RDPG. Likewise, it is seen that DDPG is more successful in cold start prediction performances. This is because DDPG can more successfully predict complex problems such as cold start with its discovery mechanism. In the MASTER framework, DDPG, LSTM, XGB Regressor, **Linear Regression (LR)**, DeepAr, **Neural Hierarchical Interpolation for Time Series Forecasting (NHITS)**, and **Teamfight Tactics (TFT)** models are compared. The results show that the XGB Regressor model has higher success than all other models. This is because ML models perform better on small data sets such as cold start. Additionally, DL models perform poorly on noisy datasets compared to ML.

- **Keeping Container Warm and Container Pool:** Most commercial platforms like AWS, and GCP keep containers alive for a certain amount of time after function execution is complete [5]. During this period, requests that may come to the server are assigned to these containers that are kept alive, and thus it is planned to prevent the occurrence of a cold start. However, this policy wastes resources as it cannot detect the exact time of cold start occurrence. Another similar method is the container pool used in the Fission and Knative

platforms. In this method, constantly running containers are used to respond to requests. This method causes a waste of resources as it will require resources to work in vain. Kumari et al. [82] proposed a new model called ACPM that provides the runtime provisioning of containers. In the two-stage model, first, the number of containers that need to be preheated is determined using the LSTM model. An integrated docker module is then used to deliver the heated containers faster and reduce the cold start frequency. In Reference [113] has been suggested to create a pool of hot containers to handle requests to the server. However, this solution is a resource-intensive process. Kumari et al. [114] proposed a new method based on hot containers. Using deep neural networks and LSTM models, the number of hot containers and the keeping times of the containers are determined. So they try to suggest a resource-aware-based approach to prevent a cold start. In another similar study, Reference [68] uses a reinforcement learning method and LSTM models to determine the number of hot containers and the time they are kept warm. One of the biggest limitations of this solution method is unnecessary resource consumption. Because, even in cases where a cold start does not occur, certain system resources are used to keep containers warm. To prevent this, AI-based container warm-up strategies can be applied.

- **Instance Prewarm-based Solutions:** It is aimed to prevent the occurrence of cold start by pre-starting the function instances required to execute the requests coming to the server. Xu et al. [115] introduced a new resource-sensitive optimization strategy to reduce the cold start frequency. First, with the **Adaptive Warm-up (AWU)** strategy, the call times of functions are estimated with a time series model. And according to this prediction result, the functions are warmed. However, these approaches may require resources to run idle, as they cannot show the same performance in sudden demand increases or determine the cold start time precisely. In this solution method, it is difficult to estimate the time that the instances need to be heated. With the increase in the number of functions, the management of pre-heated samples becomes complex. AI-based models can be leveraged to overcome these issues. Thus, for example, the heating time will be estimated more accurately.
- **Scheduling Strategies:** It is the study of container lifecycle and scaling policy to reduce cold start frequency in serverless computing. Wu et al. [116] proposed a new container lifecycle-aware scheduling strategy called CAS. CAS controls the request distribution and builds each container according to the required lifecycle stage. When scheduling a request, CAS prefers the workers with containers. And it uses the first available container for the worker of choice. In this way, it is aimed to reduce the cold start frequency. The authors of Reference [117] introduced ENSURE, a new efficient scheduler and resource manager for serverless platforms. ENSURE is designed to eliminate cold start and application resource contention using queuing concepts. ENSURE classifies function requests and flexibly scales resources based on varying workload traffic to prevent cold start. The authors of Reference [118] proposed a new system called FaasCache, which follows a GReedy-Dual-based policy to prevent cold start. GReedy-Dual is a size and frequency-sensitive caching policy. Using an analogy, the authors suggested that the keep-alive and object-caching methods could be adapted to each other. Therefore, they showed that the GReedy-Dual cache policy can be adapted to an effective keep-alive policy. The authors proposed a new container management scheme based on container sharing called Pagurus. According to this scheme, instead of starting a new container for incoming requests to the server, they appoint idle hot containers. In this way, it is aimed to prevent cold start, the time it takes to start a new container. In Reference [119], a scheduling approach called LCS is proposed to keep containers hot for longer. In this affinity-based approach, the last container used to execute incoming requests is detected. In this solution method, there are problems such as algorithm complexity that may arise when determining

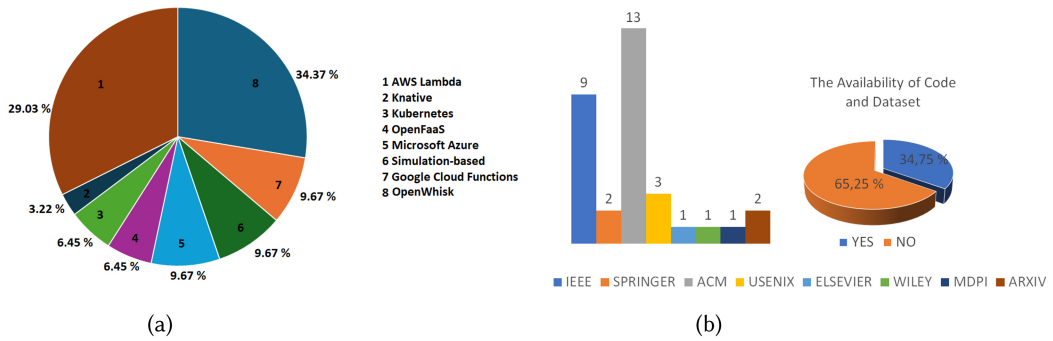


Fig. 15. (a) Percentage distribution of platforms. (b) Distribution of venues of research works and availability of code and dataset.

resource contention and scheduling strategies. To overcome these problems, adaptive scheduling algorithms and strategies to distribute the load equally to the instances can be used.

- **Container Runtime Reusing:** This method, which is similar to the container pool method to reduce the cold start frequency, uses the runtime environment instead of warm containers. Suo et al. [120] introduced the container-based runtime management framework HotC. Demand is forecasted using the exponential smoothing model and Markov chain method, and the necessary runtime environment is provided from the container runtime pool. In this way, it is tried to prevent cold start by using containers efficiently. The container remaining from previous executions should be carefully cleaned. Otherwise, it will cause security vulnerabilities. For this reason, safe container-cleaning processes must be implemented.

5.4 RQ4: Which Platforms Have Existing Studies Been Implemented?

The purpose of this research question is to investigate which serverless platforms the existing solutions for the cold start problem are applied to and evaluated. The first two authors of this article identified the platforms used by carefully examining the Abstract, Conclusion, and Implementation sections of the existing studies, respectively. Figure 15(a) shows the percentage distribution of serverless platforms on which the existing solutions reviewed in this article are implemented. As can be seen from the figure, the current studies are commercial, open-source, and simulation-based platforms, including OpenWhisk, AWS Lambda, Knative, Kubernetes, OpenFaaS, Microsoft Azure, Simulation-based, GCF, Nuclio, Firecracker, Huawei Cloud Functions, and Kubeless. The majority of existing studies [68, 83, 86, 91, 95, 106, 109, 116–118] use Openwhisk, an open-source serverless platform of 34.37%. The reasons for this are; (i) Openwhisk allows users to design their own custom runtime environments. (ii) Since Openwhisk is open source, users can easily distribute their own solutions. The second most used platform is AWS Lambda with 29.03% [82, 88, 98, 107, 109, 114, 119]. Also, AWS Lambda is the most used commercial platform compared to other commercial platforms like Microsoft and Google. This may be because AWS Lambda was the first commercial serverless platform, and hence greater adoption. Also, it may be that the infrastructure support it offers for researchers in deploying cold start solutions to the platform is more mature [5]. The third most preferred platforms are Knative [67, 91, 113], Kubernetes [108, 110, 115], and OpenFaaS [105, 114, 120] with 9.67% rates. All three technologies have flexible and efficient management and efficient resource options for developers. Microsoft Azure and simulation-based platforms account for 6.45% of platforms that cold start mitigation works implemented [90, 114, 121, 122]. Cloud Functions, Nuclio, Firecracker, Huawei Cloud Functions, and Kubeless are the least preferred platforms in the studies reviewed.

The scalability of solutions for cold start issues varies across different serverless platforms [123]. An example is a warm start. In AWS Lambda, containers continue to run for a while even after the function has completed its execution. In case of a new function call, these containers are reused to reduce the cold start latency period. Similar systems also exist in Azure and GCP. However, the performance of this solution method varies due to reasons such as cross-platform system architecture, and container startup speed [124]. Other reasons for the difference in scalability performance of solution methods across platforms are (i) Isolation Techniques and (ii) Runtime Environment [81]. Each platform uses different container isolation techniques depending on its specific architecture and this creates differences in the scalability of solutions for cold start issues [125]. AWS Lambda uses container-based solutions like “container reuse” and “keep-warm,” GCP, **VPC (Virtual Private Cloud)**, Azure, and “Durable Functions” [126]. Runtime may vary on each platform. For example, languages like Go and Node.js run faster on AWS Lambda than on Java or .NET.

5.5 RQ5: Where Are Existing Studies Published and Do They Share the Dataset or Code?

With this research question, it investigates where the studies are published and whether the dataset used in the current studies is shared with researchers. Additionally, to provide a more comprehensive evaluation, we also add evaluation information about the publication year of the article, the number of citations, and the quality of the conference/journal in Appendix C. The first two authors identified where they were published by typing the names of the articles from Google Scholar. Figure 15(b) shows the organizations where current studies are published. As can be seen, most of the works about the cold start solution in serverless computing have been published in ACM. Accessibility of the dataset is very important in terms of reproducing solutions and opening the door to new research. To answer this question, the paper’s third author first scanned the Dataset subsection. If he could not find a definitive answer to this question, then he carefully studied the entire paper and especially the reference section. Figure 15(b) shows the percentage distribution showing the accessibility of the dataset and code for the cold start papers reviewed in this article. Datasets or codes are available for only 34.75% of studies. This indicates that most of the authors focused only on design and implementation descriptions rather than the reproducibility of studies.

6 Open Challenges and Future Directions

In this systematic research article, over 100 papers were scanned to examine the cold start problem in serverless computing and categorize current solutions. Keele’s Systematic Literature Reviews method [15] was followed and the snowballing technique was applied to make a systematic review, and the 32 most meaningful and up-to-date cold start research papers were identified and classified. Based on the taxonomy study, existing studies in the literature were reviewed and compared for five Research Questions. In this section, we identify open challenges and future directions for the cold start issue still waiting to be resolved in serverless computing. A taxonomy for open challenges and future directions is provided in Figure 16. We have examined the techniques used to reduce cold start frequency and latency under two general headings in Section 5.3. The techniques generally get trade-offs such as Energy-awareness, Practicality, and Model Performance. New solutions are needed that reduce or eliminate these trade-offs. As an independent research gap from these, also energy-sensitive simulators are needed to simulate the cold start situation in serverless and serverless edge environments [37].

6.1 Energy Awareness

Some of the preferred methods for reducing cold start frequency and cold start latency require wasting resources. These methods can be listed as follows:

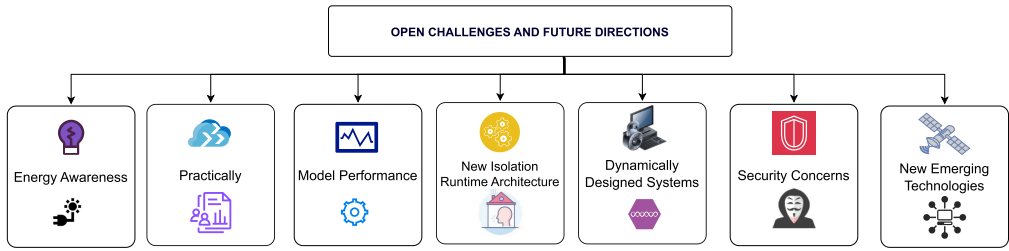


Fig. 16. Taxonomy of open challenges and future directions.

- **Periodic ping-pong-based studies.** These studies [108–110], in which periodically repetitive workloads are followed and containers are made ready on the server, often require containers to run in vain. In this solution method, new energy-aware studies can be done by using effective and high-performance AI models.
- **Studies based on Keeping Container Warm and Container Pool.** These studies [68, 82, 113, 114], which are based on the fact that the containers whose function execution is completed work for a certain period of time without scale to zero, or the use of constantly running containers, require resources to be wasted.
- **Instance Prewarm-based studies.** In these studies [115], where the function instances have been ready before the request arrives, sudden demand increases cannot be coped with, and the accuracy rates of the AI forecasting models used are not at a satisfactory level.
- **Container Runtime Reusing studies.** In these studies [120] based on the container runtime pool method, runtime environments are kept running continuously, similar to the container pool method. Keeping these runtime environments working will require resources to be wasted. There is a need for studies that provide energy optimization in the environment pool by making high-accuracy demand forecasting.
- **Cache-based studies.** Cache-based studies are based on the principle of preparing the necessary libraries in advance and quickly installing them in containers to reduce cold start latency and increase the performance of the system [86, 88–91]. However, in addition to the advantages they offer, since they are solutions that require high resource consumption, studies can be carried out that take into account various optimization aspects in this field. For example, tailored caching methods can be designed in scenarios such as applications requiring high volumes of data, taking into account applicable scenarios and load characteristics. Additionally, dynamic caching strategies can be developed to adapt to scenarios with changing workloads [127].
- **Dynamically Designed Solutions:** These are solution methods that can be dynamically adjusted to minimize the cold start latency. These methods; (i) edge-based solutions that reduce the amount of latency by moving the computing power closer to the edge of the network ([128] such as AWS Lambda, Lambda@Edge), (ii) systems that aim to reduce the amount of cold start latency by configuring the scaling speed [129], and (iii) solutions where functions are kept warm (Azure, Premium Plan [130]). All these solutions bring with them the disadvantage of unnecessary waste of resources.

6.2 Practicality

Some of the preferred methods to reduce the cold start frequency and latency still bring impractical issues such as long processing time. New practical solutions can be offered by using new optimization techniques.

- **Function Execution State-based studies.** These studies [67, 82, 105, 106], which are based on the logic of storing function execution states and using them for fast startup if needed, will take a long time to load runtime input data and execution states in scenarios where the input data payload differs. Therefore, there is a need for new studies that include scenarios where the input data payload differs.
- **Function Fusion-based studies.** In these studies [107], which aim to remove the cold start of the second function by combining sequential functions, the case of combining parallel functions still awaits to be investigated as a big challenge.

6.3 Model Performance

Some of the preferred methods to reduce cold start frequency and latency try to predict workloads and call times of functions using AI-based models [108–110, 115]. However, the AI prediction models' performances still have not reached satisfactory levels. Transfer Learning techniques and cold start datasets can be used to improve the performance of AI models. Transfer learning is a technique used in environments with limited computational capabilities and little data [131]. Reusing previously trained models for another task can increase model performance. When the literature is examined, it is seen that there is a need for an open-source dataset regarding the cold start. Models trained with datasets created from a real environment workload will give more accurate prediction results for cold start prediction. Additionally, in the future, ML models with high prediction accuracy may inspire solutions to improve cold start. ML models are expected to have higher prediction performance than DRL and DL models in small and noisy datasets such as cold start datasets.

6.4 New Isolation and Runtime Architecture

Isolation and runtime architectures are promising in solving various challenges (security, etc.), especially the cold start problem in serverless computing [132]. An example is the lightweight and secure virtualization architecture AWS Firecracker. AWS Firecracker is a microVM-based technology and offers advantages such as minimal overhead and faster operation for serverless instances compared to traditional VMs [133]. Thanks to minimal overhead, microVMs are started faster and cold start latency is directly reduced. Another example is Google's proposed gVisor technology. gVisor acts as an additional layer between the application and the processor, blocking system calls and reducing cold start latency as it provides flexibility by working compatible with existing containers [134]. Future researchers may combine lightweight microVMs and system call interception techniques to further reduce cold start latency. In addition, serious improvements can be made on the cold start time and response time of serverless computing with new techniques to be developed on container lifecycle and scalability policies. To understand the limitations of the new Isolation and runtime architecture studies carried out to reduce cold start latency, the effects of resource consumption and scalability on performance can be examined.

6.5 Dynamically Designed Serverless Computing Systems

In this subsection, the difficulties of designing serverless computing systems that can be dynamically adjusted to minimize the cold start problem in the serverless paradigm will be discussed. Almost all serverless platforms offer dynamic adjustment options to reduce cold start. The options and challenges offered by AWS Lambda, GCF, and Microsoft Azure are as follows:

- **AWS Lambda:** AWS aims to minimize the latency effect caused by cold start by processing data at the edge with Lambda@Edge technology [128]. Since the data is processed close to the data source, the response delay is reduced relative to the communication delay between

the user and the server. The other dynamic solution is to optimize the VPCs used for the lambda function. In addition to the advantages these solutions offer, they also bring additional costs and scalability problems, since edge devices have limited processing power and resources.

- Google Cloud Functions (GCF): GCF allows the scaling rate to be configured so that the amount of cold start latency can be reduced when new instances are started [129]. Additionally, GCF provides the option to adjust the number of ready instances that can meet new requests. In this way, it is aimed to prevent a cold start that may occur, as there are instances that can meet the requests in a working state. These dynamic solutions offered by GCF also have disadvantages such as cost and configuration complexity.
- Microsoft Azure: Azure tries to prevent cold start by keeping functions warm with the Premium Plan [130]. This dynamic solution has disadvantages such as unnecessary waste of resources and additional costs.

Apart from these developments, some of the promising research areas in the solutions offered regarding cold start are as follows:

- Network optimization: Promising developments that can reduce cold start in the field of network optimization can be listed as follows: (i) Edge computing: Edge computing is an emerging technology that aims to reduce latency and unnecessary bandwidth usage by bringing computing devices closer to the data source. It can be used to reduce latency caused by a cold start. The serverless edge computing paradigm, which has recently emerged by combining the serverless and edge computing paradigms, is a new research field that attracts attention in the academy [112]. (ii) **Software-defined Networking (SDN)**: It is a network management that configures network management dynamically, unlike traditional methods [135]. It can be used to dynamically route function calls in serverless computing.
- Compiler Technology: Ahead-of-time compilation, the premature compilation of code deployed for compilation in serverless computing, can be a dynamic solution to reduce cold start latency. Additionally, faster-running runtime architectures can be used to reduce cold start, as reviewed in Section 6.4.
- Resource Prediction: In addition to the ML performance improvements that predict function calls mentioned in Section 6.3, anomaly detection methods can also be used for request patterns that may require scaling.

6.6 Security Concerns

While existing cold start solutions hold great promise in optimizing latency, which is a fundamental problem in serverless computing, they still have fundamental limitations in addressing security concerns [36]. These limitations and examples are as follows:

- Lack of Inter-function Isolation: Techniques such as reusing containers whose function execution has been completed and reusing function images are some of the existing solutions. However, these solutions carry security risks by leaving the door open for hackers to obtain unsanitized data from previous functions.
- Extending the Lifecycle of Vulnerable Containers: Some cold start solution methods may increase the risk of attack for hackers as they extend the lifecycle of vulnerable containers.
- Security is the Responsibility of Cloud Providers: In a serverless platform, security is assumed to be handled by the service provider, and therefore cold start solutions do not guarantee additional protection.

Future researchers can focus on stronger isolation mechanisms and high-security cold start solutions to address security concerns such as cold start attacks in serverless computing.

6.7 New Emerging Technologies

New technologies such as edge computing and 5G can be used to reduce the negative effects of cold start issues such as latency in serverless computing. Edge computing is based on the basic principle of processing data close to the data source, reducing unnecessary bandwidth usage and latency compared to remote servers [42]. In this way, the concept of serverless edge computing, which emerged from the combination of serverless computing and edge computing, has attracted great attention [111]. However, with the widespread use of high-speed data transfer technologies such as 5G and 6G, communication will accelerate and latency periods will shorten. This will contribute greatly to the tolerance of latency periods that occur due to cold starts.

7 Conclusions

In this article, a systematic review and taxonomy study is presented for the cold start problem, which is still waiting for a solution in the serverless paradigm. First, over 100 most recent literature studies on cold start in serverless computing were collected by the authors to create a taxonomy. A systematic study was carried out by two authors to identify relevant studies that focused only on the cold start problem. And at the last stage, a paper set consisting of 32 papers was obtained by the author, who is an expert on the subject, using the snowball method. Using the paper set, the authors sought answers to five research questions regarding cold start solutions. With the first research question, the relationship between cold start and QoS was questioned. In the second research question, the factors affecting cold start latency were investigated. With the third research question, solution studies on cold start and cold start latency were classified and examined. In the fourth research question, which platforms the relevant solution methods were applied to were investigated and statistical information was obtained. In the last research question, the distribution of the journals in which the relevant research articles were published and whether the dataset/codes were shared were investigated. A classification is made of various approaches offered by academic and industrial organizations to reduce cold start frequency and shorten cold start latency in serverless computing. These approaches mainly include AI-, ML-, DL-, and RL-based approaches as well as popular techniques such as caching and application-level optimization. Existing methods are classified according to their common points as a result of the analysis. The final section discusses the open challenges of cold start in serverless computing and identifies future directions, providing valuable information for researchers. Future studies can further investigate new approaches in the future, such as integrating AI-based prediction models and caching mechanisms. Additionally, studies can be conducted to evaluate the performance of solutions proposed for different serverless platforms to improve this article.

References

- [1] Sukhpal Singh Gill, Minxian Xu, Carlo Ottaviani et al. 2022. AI for next generation computing: Emerging trends and future directions. *Internet Things* 19 (2022), 100514.
- [2] Eric Jonas, Johann Schleier-Smith, Vikram Sreekanti et al. 2019. Cloud programming simplified: A berkeley view on serverless computing. Retrieved from <https://arXiv:1902.03383>
- [3] Mario Villamizar, Oscar Garces, Lina Ochoa et al. 2016. Infrastructure cost comparison of running web applications in the cloud using AWS lambda and monolithic and microservice architectures. In *Proceedings of the 16th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGrid'16)*. IEEE, 179–182.
- [4] M. Golec, S. S. Gill, A. K. Parlikad, and S. Uhlig. 2023. HealthFaaS: Ai-based smart healthcare system for heart patients using serverless computing. *IEEE Internet of Things Journal* 10, 21 (2023), 18469–18476.
- [5] J. Wen, Z. Chen, X. Jin, and X. Liu. 2023. Rise of the planet of serverless computing: A systematic review. *ACM Transactions on Software Engineering and Methodology* 32, 5 (2023), 1–61.
- [6] Baotong Chen, Jiafu Wan, Antonio Celesti, Di Li, Haider Abbas, and Qin Zhang. 2018. Edge computing in IoT-based manufacturing. *IEEE Commun. Mag.* 56, 9 (2018), 103–109.

- [7] Renchao Xie, Qinqin Tang, Shi Qiao, Han Zhu, F. Richard Yu, and Tao Huang. 2021. When serverless computing meets edge computing: Architecture, challenges, and open issues. *IEEE Wireless Commun.* 28, 5 (2021), 126–133.
- [8] Luciano Baresi and Danilo Filgueira Mendonça. 2019. Towards a serverless platform for edge computing. In *Proceedings of the IEEE International Conference on Fog Computing (ICFC'19)*. IEEE, 1–10.
- [9] Markets and Markets. ([n.d.]). Retrieved from <https://www.marketsandmarkets.com/Market-Reports/edge-computing-market-133384090.html>
- [10] Muhammed Golec, Ridvan Ozturac, Zahra Pooranian, Sukhpal Singh Gill, and Rajkumar Buyya. 2021. IFaaSBus: A security-and-privacy-based lightweight framework for serverless computing using IoT and machine learning. *IEEE Trans. Industr. Info.* 18, 5 (2021), 3522–3529.
- [11] Paul Castro, Vatche Ishakian, Vinod Muthusamy, and Aleksander Slominski. 2019. The rise of serverless computing. *Commun. ACM* 62, 12 (2019), 44–54.
- [12] I. Baldini, P. Castro, K. Chang, P. Cheng, S. Fink, V. Ishakian, and P. Suter. 2017. Serverless computing: Current trends and open problems. *Research Advances in Cloud Computing*. 1–20.
- [13] Jeongchul Kim and Kyungyong Lee. 2019. Practical cloud workloads for serverless FaaS. In *Proceedings of the ACM Symposium on Cloud Computing*. 477–477.
- [14] Muhammed Golec, Sundas Iftikhar, Pratibha Prabhakaran, Sukhpal Singh Gill, and Steve Uhlig. 2023. QoS analysis for serverless computing using machine learning. In *Serverless Computing: Principles and Paradigms*. Springer, 175–192.
- [15] S. Keele. 2007. Guidelines for performing systematic literature reviews in software engineering (Vol. 5). Technical report, ver. 2.3 ebse technical report. ebse. https://www.researchgate.net/profile/Barbara-Kitchenham/publication/302924724_Guidelines_for_performing_Systematic_Literature_Reviews_in_Software_Engineering/links/61712932766c4a211c03a6f7/Guidelines-for-performing-Systematic-Literature-Reviews-in-Software-Engineering.pdf
- [16] Parichehr Vahidinia, Bahar Farahani, and Fereidoon Shams Aliee. 2020. Cold start in serverless computing: Current trends and mitigation strategies. In *Proceedings of the International Conference on Omni-layer Intelligent Systems*. IEEE, 1–7.
- [17] Anupama Mampage, Shanika Karunasekera, and Rajkumar Buyya. 2022. A holistic view on resource management in serverless computing environments: Taxonomy and future directions. *Comput. Surveys* 54, 11s (2022), 1–36.
- [18] Zijun Li, Linsong Guo, Jiagan Cheng, Quan Chen, BingSheng He, and Minyi Guo. 2022. The serverless computing survey: A technical primer for design architecture. *ACM Comput. Surveys* 54, 10s (2022), 1–34.
- [19] Hossein Shafiei, Ahmad Khonsari, and Payam Mousavi. 2022. Serverless computing: A survey of opportunities, challenges, and applications. *Comput. Surveys* 54, 11s (2022), 1–32.
- [20] Simon Eismann, Joel Scheuner et al. 2021. The state of serverless applications: Collection, characterization, and community consensus. *IEEE Trans. Softw. Eng.* 48, 10 (2021), 4152–4166.
- [21] S. N. A. Jawaddi and A. Ismail. 2023. Autoscaling in serverless computing: Taxonomy and OpenChallenges. <https://www.researchsquare.com/article/rs-2897886/v1>
- [22] Hassan B. Hassan, Saman A. Barakat, and Qusay I. Sarhan. 2021. Survey on serverless computing. *J. Cloud Comput.* 10, 1 (2021), 1–29.
- [23] Yongkang Li, Yanying Lin, Yang Wang, Kejiang Ye, and Chengzhong Xu. 2022. Serverless computing: state-of-the-art, challenges and opportunities. *IEEE Trans. Services Comput.* 16, 2 (2022), 1522–1539.
- [24] S. Eismann, J. Scheuner, E. V. Eyk, M. Schwinger, J. Grohmann, N. R. Herbst, C. L. Abad, and A. Iosup. A review of serverless use cases and their characteristics. Retrieved from <https://arXiv:2008.11110>
- [25] Gustavo André Setti Cassel, Vinicius Facco Rodrigues, Rodrigo da Rosa Righi, Marta Rosecler Bez, Andressa Cruz Nepomuceno, and Cristiano André da Costa. 2022. Serverless computing for Internet of Things: A systematic literature review. *Future Gen. Comput. Syst.* 128 (2022), 299–316.
- [26] Eduard Marin, Diego Perino, and Roberto Di Pietro. 2022. Serverless computing: A security perspective. *J. Cloud Comput.* 11, 1 (2022), 1–12.
- [27] Raghava Satya Saikrishna Dittakavi. 2023. Cold start latency in serverless computing: Current trends and mitigation techniques. *Eduzone: Int. Peer Rev./Ref. Multidisc. J.* 12, 2 (2023), 135–139.
- [28] Md Abu Imran Mallick and Rishab Nath. Securing the server-less frontier: Challenges and innovative solutions in network security for server-less computing. *Reading Time* 2024 ([n.d.]), 4–15.
- [29] Sina Ahmadi. 2024. Challenges and solutions in network security for serverless computing. *Int. J. Curr. Sci. Res. Rev.* 7, 01 (2024), 218–229.
- [30] Praveen Kumar Donta, Ilir Murturi, Victor Casamayor Pujol, Boris Sedlak, and Schahram Dustdar. 2023. Exploring the potential of distributed computing continuum systems. *Computers* 12, 10 (2023), 198.
- [31] Praveen Kumar Donta and Schahram Dustdar. 2023. Towards intelligent data protocols for the edge. In *Proceedings of the IEEE International Conference on Edge Computing and Communications (EDGE'23)*. IEEE, 372–380.

- [32] Xin-Lin Huang, Xiaomin Ma, and Fei Hu. 2018. Machine learning and intelligent communications. *Mobile Netw. Appl.* 23 (2018), 68–70.
- [33] Maziyar Nazari, Sepideh Goodarzy, Eric Keller, Eric Rozner, and Shivakant Mishra. 2021. Optimizing and extending serverless platforms: A survey. In *Proceedings of the 8th International Conference on Software Defined Systems (SDS'21)*. IEEE, 1–8.
- [34] Raghubir Singh and Sukhpal Singh Gill. 2023. Edge AI: A survey. *Internet Things Cyber-Phys. Syst.* 3 (2023), 71–92.
- [35] Babar Ali, Muhammed Golec, Sukhpal Singh Gill, Huaming Wu, Felix Cuadrado, and Steve Uhlig. 2024. EdgeBus: Co-Simulation based resource management for heterogeneous mobile edge computing environments. *Internet Things* 28 (2024), 101368.
- [36] Surendra Kumar, Mridula Dwivedi et al. 2024. A comprehensive review of vulnerabilities and AI-enabled defense against DDoS attacks for securing cloud services. *Comput. Sci. Rev.* 53 (2024), 100661.
- [37] Aadharsh Roshan Nandhakumar, Ayush Baranwal et al. 2024. Edgeaisim: A toolkit for simulation and modelling of ai models in edge computing environments. *Measure.: Sensors* 31 (2024), 100939.
- [38] Sujit Bebortta, Saneev Kumar Das, Meenakshi Kandpal et al. 2020. Geospatial serverless computing: Architectures, tools and future directions. *ISPRS International J. Geo-Info.* 9, 5 (2020), 311.
- [39] Muhammed Oguzhan Mete and Tahsin Yomralioglu. 2021. Implementation of serverless cloud GIS platform for land valuation. *Int. J. Dig. Earth* 14, 7 (2021), 836–850.
- [40] Justin Franz, Tanmayi Nagasuri, Andrew Wartman, Agnese V. Ventrella, and Flavio Esposito. 2018. Reunifying families after a disaster via serverless computing and raspberry pi. In *Proceedings of the IEEE International Symposium on Local and Metropolitan Area Networks (LANMAN'18)*. IEEE, 131–132.
- [41] Luis Felipe Herrera-Quintero, Julian Camilo Vega-Alfonso, Klaus Bodo Albert Banse, and Eduardo Carrillo Zambrano. 2018. Smart ITS sensor for the transportation planning based on IoT approaches using serverless and microservices architecture. *IEEE Intell. Transport. Syst. Mag.* 10, 2 (2018), 17–27.
- [42] Muhammed Golec and Sukhpal Singh Gill. 2024. Computing: Looking back and moving forward. In *Proceedings of the 21st International Conference on Smart Business Technologies (ICSBT'24)* (2024), 7–14.
- [43] Steve Jefferson, Pethuru Chelliah, and Chellammal Surianarayanan. 2022. A resource-optimized and accelerated sentiment analysis method using serverless computing. *Procedia Comput. Sci.* 215 (2022), 33–43.
- [44] Moslem Dehghani, Mohammad Ghiasi, Taher Niknam, Abdollah Kavousi-Fard, Mokhtar Shasadeghi, Noradin Ghadimi, and Farhad Taghizadeh-Hesary. 2020. Blockchain-based securing of data exchange in a power transmission system considering congestion management and social welfare. *Sustainability* 13, 1 (2020), 90.
- [45] Muhammed Golec, Deepraj Chowdhury, Shivam Jaglan, Sukhpal Singh Gill, and Steve Uhlig. 2022. Aiblock: Blockchain based lightweight framework for serverless computing using ai. In *Proceedings of the 22nd IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid'22)*. IEEE, 886–892.
- [46] Babar Ali, Muhammed Golec, Sukhpal Singh Gill, Felix Cuadrado, and Steve Uhlig. 2024. ProKube: Proactive kubernetes orchestrator for inference in heterogeneous edge computing. *Int. J. Netw. Manage.* (2024), e2298.
- [47] Shajulin Benedict. 2020. Serverless blockchain-enabled architecture for IoT societal applications. *IEEE Trans. Comput. Soc. Syst.* 7, 5 (2020), 1146–1158.
- [48] B. Sedlak, V. C. Pujol, P. K. Donta, and S. Dustdar. 2024. Equilibrium in the computing continuum through active inference. *Future Generation Computer Systems*.
- [49] Nima Mahmoudi, Changyuan Lin, Hamzeh Khazaei, and Marin Litoiu. 2019. Optimizing serverless computing: Introducing an adaptive function placement algorithm. In *Proceedings of the 29th Annual International Conference on Computer Science and Software Engineering*. 203–213.
- [50] Victor Casamayor Pujol, Boris Sedlak, Praveen Kumar Donta, and Schahram Dustdar. 2024. On causality in distributed continuum systems. *IEEE Internet Comput.* 28, 2 (2024), 57–64.
- [51] Donghee Shin. 2015. Beyond user experience of cloud service: Implication for value sensitive approach. *Telemat. Informat.* 32, 1 (2015), 33–44.
- [52] Gaetano Somma, Constantine Ayimba, Paolo Casari, Simon Pietro Romano, and Vincenzo Mancuso. 2020. When less is more: Core-restricted container provisioning for serverless computing. In *Proceedings of the IEEE Conference on Computer Communications Workshops (INFOCOM'20)*. IEEE, 1153–1159.
- [53] Sukhpal Singh Gill, Muhammed Golec, Jianmin Hu et al. 2024. Edge AI: A taxonomy, systematic review and future directions. Technical Report, Retrieved from <https://arXiv:2407.04053>
- [54] Roberto Morabito, Vittorio Cozzolino, Aaron Yi Ding, Nicklas Beijar, and Jorg Ott. 2018. Consolidate IoT edge computing with lightweight virtualization. *IEEE Netw.* 32, 1 (2018), 102–111.
- [55] R. Krishnamurthi, A. Kumar, S. S. Gill, and R. Buyya. 2023. Serverless Computing: New trends and research directions. *Serverless Computing: Principles and Paradigms*. 1–13.
- [56] Simon Eismann, Joel Scheuner, Erwin Van Eyk, Maximilian Schwinger, Johannes Grohmann, Nikolas Herbst, Cristina L. Abad, and Alexandru Iosup. 2020. Serverless applications: Why, when, and how? *IEEE Softw.* 38, 1 (2020), 32–39.

- [57] Parminder Singh et al. 2022. Machine learning for cloud, fog, edge and serverless computing environments: Comparisons, performance evaluation benchmark and future directions. *Int. J. Grid Util. Comput.* 13, 4 (2022), 447–457.
- [58] Nabil El Ioini, David Hästbacka, Claus Pahl, and Davide Taibi. 2021. Platforms for serverless at the edge: A review. In *Proceedings of the International Workshops on Advances in Service-Oriented and Cloud Computing (ESOCC'20)*. Springer, 29–40.
- [59] Mohammad S. Aslanpour, Adel N. Toosi, Claudio Cicconetti et al. 2021. Serverless edge computing: Vision and challenges. In *Proceedings of the Australasian Computer Science Week Multiconference*. 1–10.
- [60] Mirela-Cristina Voicu and Alina-Mihaela Babonea. 1997. Using the snowball method in marketing research on hidden populations. *Soc. Prob.* 44, 2 (1997), 1341–1351.
- [61] S. Iftikhar, S. S. Gill, C. Song, M. Xu, M. S. Aslanpour, A. N. Toosi, and S. Uhlig. 2023. AI-based fog and edge computing: A systematic review, taxonomy and future directions. *Internet of Things* 21 (2023), 100674.
- [62] Barbara Kitchenham. 2004. Procedures for performing systematic reviews. *Keele University* 33, 2004 (2004), 1–26.
- [63] Kwangkyyu Lee, Jinhee Park, and Jongmoon Baik. 2015. Location-based web service QoS prediction via preference propagation for improving cold start problem. In *Proceedings of the IEEE International Conference on Web Services*. IEEE, 177–184.
- [64] Zhen Chen, Limin Shen, Feng Li, and Dianlong You. 2017. Your neighbors alleviate cold-start: On geographical neighborhood influence to collaborative web service QoS prediction. *Knowl.-Based Syst.* 138 (2017), 188–201.
- [65] Sasko Ristov, Marjan Gusev, and Magdalena Kostoska. 2012. A new methodology for security evaluation in cloud computing. In *Proceedings of the 35th International Convention MIPRO*. IEEE, 1484–1489.
- [66] Daniel Bardsley, Larry Ryan, and John Howard. 2018. Serverless performance and optimization strategies. In *Proceedings of the IEEE International Conference on Smart Cloud (SmartCloud'18)*. IEEE, 19–26.
- [67] Dmitrii Ustiugov, Plamen Petrov, Marios Kogias, Edouard Bugnion, and Boris Grot. 2021. Benchmarking, analysis, and optimization of serverless function snapshots. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 559–572.
- [68] Parichehr Vahidinia, Bahar Farahani, and Fereidoon Shams Aliee. 2022. Mitigating cold start problem in serverless computing: A reinforcement learning approach. *IEEE Internet Things J.* 10, 5 (2022), 3917–3927.
- [69] István Pelle, János Czentye, János Dóka, and Balázs Sonkoly. 2019. Towards latency sensitive cloud native applications: A performance study on AWS. In *Proceedings of the 12th International Conference on Cloud Computing*. IEEE, 272–280.
- [70] Shanshan Chen, Xiaoxin Tang, Hongwei Wang, Han Zhao, and Minyi Guo. 2016. Towards scalable and reliable in-memory storage system: A case study with redis. In *Proceedings of the IEEE Trustcom/BigDataSE/ISPA*. IEEE, 1660–1667.
- [71] Thilina Gunarathne, Judy Qiu, and Geoffrey Fox. 2011. Iterative mapreduce for azure cloud. In *Proceedings of the Conference on Cloud Computing and Its Applications (CCA'11)*. 12–13.
- [72] C. Kavitha and X. Anita. 2020. Task failure resilience technique for improving the performance of MapReduce in Hadoop. *ETRI J.* 42, 5 (2020), 748–760.
- [73] Chinmay Kulkarni, Badrish Chandramouli, and Ryan Stutsman. 2020. Achieving high throughput and elasticity in a larger-than-memory store. Retrieved from <https://arXiv:2006.03206>
- [74] Dinesh Agarwal and Sushil K. Prasad. 2012. Azurebench: Benchmarking the storage services of the azure cloud platform. In *Proceedings of the 26th International Parallel and Distributed Processing Symposium Workshops and Ph.D. Forum*. 1048–1057.
- [75] Siddharth Agarwal, Maria A. Rodriguez, and Rajkumar Buyya. 2021. A reinforcement learning approach to reduce serverless function cold start frequency. In *Proceedings of the IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid'21)*. IEEE, 797–803.
- [76] David Jackson and Gary Clynch. 2018. An investigation of the impact of language runtime on the performance and cost of serverless functions. In *Proceedings of the IEEE/ACM International Conference on Utility and Cloud Computing Companion (UCC Companion)*. IEEE, 154–160.
- [77] Johannes Manner, Martin Endreß, Tobias Heckel, and Guido Wirtz. 2018. Cold start influencing factors in function as a service. In *Proceedings of the IEEE/ACM International Conference on Utility and Cloud Computing Companion (UCC Companion)*. IEEE, 181–188.
- [78] Maciej Pawlik, Kamil Figiela, and Maciej Malawski. 2018. Performance evaluation of parallel cloud functions. In *Proceedings of the International Conference on Parallel Processing (ICPP'18)*.
- [79] Liang Wang, Mengyuan Li, Yinqian Zhang, Thomas Ristenpart, and Michael Swift. 2018. Peeking behind the curtains of serverless platforms. In *Proceedings of the USENIX Annual Technical Conference (USENIX ATC'18)*. 133–146.
- [80] Mehrshad Hosseini and Omid Sahragard. 2019. Aws lambda language performance. (2019).
- [81] Hyungro Lee, Kumar Satyam, and Geoffrey Fox. 2018. Evaluation of production serverless computing environments. In *Proceedings of the IEEE 11th International Conference on Cloud Computing (CLOUD'18)*. IEEE, 442–450.

- [82] A. Kumari, and B. Sahoo. 2024. ACPM: adaptive container provisioning model to mitigate serverless cold-start. *Cluster Computing* 27, 2 (2023), 1333–1360.
- [83] Istemi Ekin Akkus, Ruichuan Chen, Ivica Rimac, Manuel Stein, Klaus Satzke, Andre Beck, Paarijaat Aditya, and Volker Hilt. 2018. SAND: Towards High-Performance serverless computing. In *Proceedings of the Usenix Annual Technical Conference (USENIX ATC'18)*. 923–935.
- [84] Kostis Kaffes, Neeraja J. Yadwadkar, and Christos Kozyrakis. 2019. Centralized core-granular scheduling for serverless functions. In *Proceedings of the ACM Symposium on Cloud Computing*. 158–164.
- [85] Dong Du, Tianyi Yu, Yubin Xia et al. 2020. Catalyzer: Sub-millisecond startup for serverless computing with initialization-less booting. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems*. 467–481.
- [86] Anup Mohan, Harshad Sane, Kshitij Doshi, Saikrishna Edupuganti, Naren Nayak, and Vadim Sukhomlinov. 2019. Agile cold starts for scalable serverless. In *Proceedings of the 11th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud'19)*.
- [87] Asfandiyar Qureshi. 2010. *Power-demand routing in massive geo-distributed systems*. Ph.D. Dissertation. Massachusetts Institute of Technology.
- [88] X. Liu, J. Wen, Z. Chen, D. Li, J. Chen, Y. Liu, and X. Jin. 2023. Faaslight: General application-level cold-start latency optimization for function-as-a-service in serverless computing. *ACM Transactions on Software Engineering and Methodology* 32, 5 (2023), 1–29.
- [89] Kai-Ting Amy Wang, Rayson Ho, and Peng Wu. 2019. Replayable execution optimized for page sharing for a managed runtime environment. In *Proceedings of the 14th EuroSys Conference 2019*. 1–16.
- [90] Pawel Zuk and Krzysztof Rzdca. 2020. Scheduling methods to reduce response latency of function as a service. In *Proceedings of the IEEE 32nd International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD'20)*. IEEE, 132–140.
- [91] Nilanjan Daw, Umesh Bellur, and Purushottam Kulkarni. 2020. Xanadu: Mitigating cascading cold starts in serverless function chain deployments. In *Proceedings of the 21st International Middleware Conference*. 356–370.
- [92] Jingyuan Zhang, Ao Wang, Xiaolong Ma, Benjamin Carver et al. 2022. Infinistore: Elastic serverless cloud storage. Retrieved from <https://arxiv:2209.01496>
- [93] Priyanka Vergadia. 2022. *Visualizing Google Cloud: 101 Illustrated References for Cloud Engineers and Architects*. John Wiley & Sons.
- [94] Ankit Kumar. 2022. *Using Redis for persistent storage in serverless architecture to maintain state management*. Ph.D. Dissertation. Dublin, National College of Ireland.
- [95] Adam Hall and Umakishore Ramachandran. 2019. An execution model for serverless functions at the edge. In *Proceedings of the International Conference on Internet of Things Design and Implementation*. 225–236.
- [96] Chetankumar Mistry, Bogdan Stelea, Vijay Kumar, and Thomas Pasquier. 2020. Demonstrating the practicality of unikernels to build a serverless platform at the edge. In *Proceedings of the IEEE International Conference on Cloud Computing Technology and Science (CloudCom'20)*. IEEE, 25–32.
- [97] Phani Kishore Gadepalli, Sean McBride, Gregor Peach, Ludmila Cherkasova, and Gabriel Parmer. 2020. Sledge: A serverless-first, light-weight Wasm runtime for the edge. In *Proceedings of the 21st International Middleware Conference*. 265–279.
- [98] Khondokar Solaiman and Muhammad Abdullah Adnan. 2020. WLEC: A not so cold architecture to mitigate cold start problem in serverless computing. In *Proceedings of the International Conference on Cloud Engineering*. IEEE, 144–153.
- [99] Edward Oakes, Leon Yang, Dennis Zhou, Kevin Houck, Tyler Harter, Andrea Arpaci-Dusseau, and Remzi Arpaci-Dusseau. 2018. SOCK: Rapid task provisioning with Serverless-Optimized containers. In *Proceedings of the USENIX Annual Technical Conference (USENIX ATC'18)*. 57–70.
- [100] Zijun Li, Jiagan Cheng, Quan Chen, Eryu Guan, Zizheng Bian, Yi Tao, Bin Zha, Qiang Wang, Weidong Han, and Minyi Guo. 2022. RunD: A lightweight secure container runtime for high-density deployment and high-concurrency startup in serverless computing. In *Proceedings of the USENIX Annual Technical Conference (USENIX ATC'22)*. 53–68.
- [101] Simon Shillaker and Peter Pietzuch. 2020. Faasm: Lightweight isolation for efficient stateful serverless computing. In *Proceedings of the USENIX Annual Technical Conference (USENIX ATC'20)*. 419–433.
- [102] Kapil Netaji Vhatkar and Girish P. Bhole. 2020. Particle swarm optimisation with grey wolf optimisation for optimal container resource allocation in cloud. *IET Netw.* 9, 4 (2020), 189–199.
- [103] Jingyuan Hu, Xiaokuang Bai, Sai Sha, Yingwei Luo, Xiaolin Wang, and Zhenlin Wang. 2018. HUB: Hugepage ballooning in kernel-based virtual machines. In *Proceedings of the International Symposium on Memory Systems*. 31–37.
- [104] Samuli Ylenius. 2020. *Mitigating JavaScript's overhead with WebAssembly*. Master's Thesis, Tampere University.
- [105] Paulo Silva, Daniel Fireman, and Thiago Emmanuel Pereira. 2020. Prebaking functions to warm the serverless cold start. In *Proceedings of the 21st International Middleware Conference*. 1–13.

- [106] James Cadden, Thomas Unger, Yara Awad, Han Dong, Orran Krieger, and Jonathan Appavoo. 2020. SEUSS: Skip redundant paths to make serverless fast. In *Proceedings of the 15th European Conference on Computer Systems*. 1–15.
- [107] Seungjun Lee, Daegun Yoon, Sangho Yeo, and Sangyoon Oh. 2021. Mitigating cold start problem in serverless computing with function fusion. *Sensors* 21, 24 (2021), 8416.
- [108] Jashwant Raj Gunasekaran, Prashanth Thinakaran, Nachiappan Chidambaram, Mahmut T. Kandemir, and Chita R. Das. 2020. Fifer: Tackling underutilization in the serverless era. Retrieved from <https://arXiv:2008.12819>
- [109] David Bermbach, Ahmet-Serdar Karakaya, and Simon Buchholz. 2020. Using application knowledge to reduce cold starts in FaaS services. In *Proceedings of the 35th Annual ACM Symposium on Applied Computing*. 134–143.
- [110] Shay Horovitz, Roei Amos, Ohad Baruch, Tomer Cohen, Tal Oyar, and Afik Deri. 2019. Faastest-machine learning based cost and performance faas optimization. In *Proceedings of the 15th International Conference on Economics of Grids, Clouds, Systems, and Services (GECON'18)*. Springer, 171–186.
- [111] M. Golec, S. S. Gill, F. Cuadrado, A. K. Parlikad, M. Xu, H. Wu, and S. Uhlig. 2023. Atom: Ai-powered sustainable resource management for serverless edge computing environments. *IEEE Transactions on Sustainable Computing*.
- [112] M. Golec, S. S. Gill, H. Wu, T. C. Can, M. Golec, O. Cetinkaya, and S. Uhlig. 2024. Master: Machine learning-based cold start latency prediction framework in serverless edge computing environments for industry 4.0. *IEEE Journal of Selected Areas in Sensors*.
- [113] Ping-Min Lin and Alex Glikson. 2019. Mitigating cold starts in serverless platforms: A pool-based approach. Retrieved from <https://arXiv:1903.12221>
- [114] Anisha Kumari, Bibhudatta Sahoo, and Ranjan Kumar Behera. 2022. Mitigating cold-start delay using warm-start containers in serverless platform. In *Proceedings of the IEEE 19th India Council International Conference (INDICON'22)*. IEEE, 1–6.
- [115] Zhengjun Xu, Haitao Zhang, Xin Geng, Qiong Wu, and Huadong Ma. 2019. Adaptive function launching acceleration in serverless computing platforms. In *Proceedings of the IEEE 25th International Conference on Parallel and Distributed Systems (ICPADS'19)*. IEEE, 9–16.
- [116] Song Wu, Zhiheng Tao, Hao Fan, Zhuo Huang, Xinmin Zhang, Hai Jin, Chen Yu, and Chun Cao. 2022. Container lifecycle-aware scheduling for serverless computing. *Software: Pract. Exper.* 52, 2 (2022), 337–352.
- [117] Amoghavarsha Suresh, Gagan Somashekar et al. 2020. Ensure: Efficient scheduling and autonomous resource management in serverless environments. In *Proceedings of the IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS'20)*. IEEE, 1–10.
- [118] Alexander Fuerst and Prateek Sharma. 2021. FaasCache: Keeping serverless computing alive with greedy-dual caching. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 386–400.
- [119] Biswajeet Sethi, Sourav Kanti Addya, and Soumya K. Ghosh. 2023. LCS: Alleviating total cold start latency in serverless applications with LRU warm container approach. In *Proceedings of the 24th International Conference on Distributed Computing and Networking*. 197–206.
- [120] Kun Suo, Junggab Son, Dazhao Cheng, Wei Chen, and Sabur Baidya. 2021. Tackling cold start of serverless applications by efficient and adaptive container runtime reusing. In *Proceedings of the IEEE International Conference on Cluster Computing (CLUSTER'21)*. IEEE, 433–443.
- [121] Zijun Li, Linsong Guo, Quan Chen, Jiagan Cheng, Chuhao Xu, Deze Zeng, Zhuo Song, Tao Ma, Yong Yang, Chao Li et al. 2022. Help rather than recycle: Alleviating cold startup in serverless computing through Inter-function container sharing. In *Proceedings of the USENIX Annual Technical Conference (USENIX ATC'22)*. 69–84.
- [122] Alim Ul Gias and Giuliano Casale. 2020. Cocoa: Cold start aware capacity planning for function-as-a-service platforms. In *Proceedings of the 28th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS'20)*. IEEE, 1–8.
- [123] Nicollas Silva, Diego Carvalho, Adriano C. M. Pereira, Fernando Mourão, and Leonardo Rocha. 2019. The pure cold-start problem: A deep study about how to conquer first-time users in recommendations domains. *Info. Syst.* 80 (2019), 1–12.
- [124] Garrett McGrath and Paul R. Brenner. 2017. Serverless computing: Design, implementation, and performance. In *Proceedings of the IEEE 37th International Conference on Distributed Computing Systems Workshops (ICDCSW'17)*. IEEE, 405–410.
- [125] Alfonso Pérez, Germán Moltó, Miguel Caballer, and Amanda Calatrava. 2018. Serverless computing for container-based architectures. *Future Gen. Comput. Syst.* 83 (2018), 50–59.
- [126] Zhenke Chen, He Li, Kaoru Ota, and Mianxiong Dong. 2023. HyScaler: A dynamic, hybrid VNF scaling system for building elastic service function chains across multiple servers. *IEEE Trans. Netw. Serv. Manage.* 20, 4 (2023), 4803–4814.
- [127] A. Souri, S. E. Mood, M. Gao, and K. C. Li. 2024. Tournament based equilibrium optimization for minimizing energy consumption on dynamic task scheduling in cloud-edge computing. *Cluster Computing*. 1–13.

- [128] Jacob Koch and Wei Hao. 2021. An empirical study in edge computing using AWS. In *Proceedings of the IEEE 11th Annual Computing and Communication Workshop and Conference (CCWC'21)*. IEEE, 0542–0549.
- [129] Vitthal Srinivasan, Janani Ravi, and Judy Raj. 2018. *Google Cloud Platform for Architects: Design and Manage Powerful Cloud Solutions*. Packt Publishing Ltd.
- [130] Florin Olariu and Lenuța Alboaie. 2023. Challenges in optimizing migration costs from on-premises to microsoft azure. *Procedia Comput. Sci.* 225 (2023), 3649–3659.
- [131] Deepraj Chowdhury, Anik Das, Ajoy Dey et al. 2023. CoviDetector: A transfer learning-based semi supervised approach to detect COVID-19 using CXR images. *BenchCouncil Trans. Bench. Stand. Eval.* 3, 2 (2023), 100119.
- [132] Sukhpal Singh Gill and Rajkumar Buyya. 2024. Transforming research with quantum computing. *J. Econ. Technol.* 2 (2024), 1–8.
- [133] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. 2020. Firecracker: Lightweight virtualization for serverless applications. In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI'20)*. 419–434.
- [134] Ethan G. Young, Pengfei Zhu, Tyler Caraza-Harter, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2019. The true cost of containing: A gVisor case study. In *Proceedings of the 11th USENIX Workshop on Hot Topics in Cloud Computing*.
- [135] Fatemeh Banaie and Karim Djemame. 2022. A serverless computing platform for software defined networks. In *Proceedings of the International Conference on the Economics of Grids, Clouds, Systems, and Services*. Springer, 113–123.

Received 13 October 2023; revised 29 September 2024; accepted 12 October 2024